A Survey on Continuous Diffusion and Flow Language Models

Sophia Tang[1], Chanhyuk Lee[2], Floor Eijkelboom[3], Jinwoo Kim[2]

[1]University of Pennsylvania [2]KAIST [3]University of Amsterdam

Project website | Paper collection

Abstract. Continuous diffusion and flow language models generate text by transforming noise into continuous representations of tokens. They can refine many positions in parallel while preserving dependencies between them, and their continuous states allow generation to be steered using gradients of a differentiable objective. Joint generation is especially useful in the few-step regime, where standard parallel discrete-diffusion samplers can incur a factorization error by sampling multiple tokens conditionally independently. Recent advances have made continuous models competitive with discrete diffusion on language benchmarks and have enabled scaling to billions of parameters. This survey explains the principles behind these advances and organizes the field by how models represent text, learn generative dynamics, and generate samples. We examine the methods that enable few-step generation, including flow maps and distillation, and connect them to developments in image and video generation. We then review scaling, applications, and evaluation, and identify open questions in representation learning, scaling, few-step distillation, post-training, multimodality, and safety.

Index Terms: Deep learning, large language model, continuous language model, continuous diffusion language model, continuous flow language model, diffusion model, flow matching

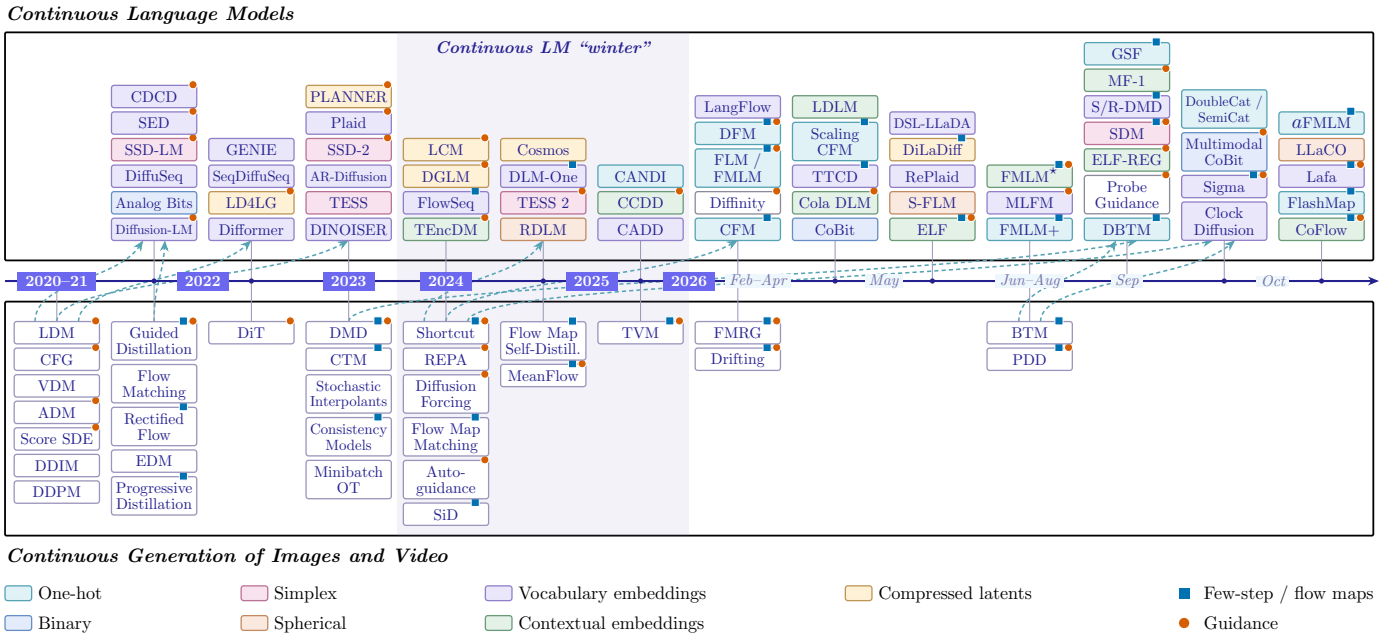


Fig. 1: Advances in continuous language models (top) and in foundations of continuous generation of images and video (bottom).

1 Introduction

Language models have become central to artificial intelligence, supporting applications in writing, programming, and problem solving. Most are *autoregressive* (AR): they predict the next token from the tokens that precede it, then repeat this process to extend the sequence. This left-to-right factorization makes sequence likelihood straightforward to compute and yields an effective next-token prediction objective, enabling models with broad capabilities [1, 2]. At generation time, however, each token depends on previously generated tokens, making standard decoding sequential and slow.

Diffusion and flow models offer an alternative. They generate a sequence by refining a noisy state, allowing many

E-mail: sophtang@engineering.upenn.edu, chanhyuk-lee@kaist.ac.kr f.eijkelboom@uva.nl, jinwoo-kim@kaist.ac.kr

token positions to be updated together. The success of these models in image and video generation has motivated two approaches to language. *Discrete* models operate directly on token sequences. Masked diffusion models generate text by iteratively filling masked positions [3–5], whereas uniform diffusion models reverse a corruption process that replaces tokens with ones drawn uniformly from the vocabulary, allowing tokens to be revised throughout generation [6, 7]. Furthermore, discrete flow models learn the generating probability velocity of a discrete jump process that moves tokens from a source distribution to the data distribution [8, 9]. By contrast, *continuous* models represent text with vectors and learn to transform noise into these representations before decoding them into tokens [10–12]. Early continuous models faced difficulties with representation learning, training objectives, and noise schedules, while discrete diffusion offered a simpler prediction problem and achieved stronger results at scale [13, 14].

Beginning in 2026, this picture has changed substantially. Continuous language models now match or surpass strong discrete baselines on generation benchmarks [15–17], and have been scaled to billions of parameters [18–20]. A key advantage emerges in one-step and few-step generation. Standard parallel discrete-diffusion models update tokens conditionally independently given the current state, so parallel updates can discard dependencies even when the predicted marginals are correct. Since a continuous model generates tokens through a joint transformation of noise, it can capture dependencies among tokens generated in parallel. This transformation can be learned directly as a flow map [21], enabling coherent generation with far fewer network evaluations [15, 22–24]. Continuous states also make gradient-based guidance possible and allow language models to use sampling and distillation methods developed for images and video. Figure 1 traces these developments and their connections.

This survey gives a unified explanation of continuous diffusion and flow language models, complementing broader surveys and perspectives of diffusion language modeling [25–27]. Our core focus is generation through continuous diffusion or flow dynamics. We also discuss hybrid latent models and direct continuous-noise generators that share this interface, including applications to biological sequences. We begin with the motivations and mathematical foundations in Section 2, then cover model design, inference, and few-step generation in Sections 3–5. Sections 6 and 7 examine scaling, applications, and evaluation. Section 8 identifies the main open problems. Figure 2 summarizes this organization.[1]

2 Motivations and Preliminaries

Continuous representations are useful for language generation for two main reasons: they allow many tokens to be generated jointly while capturing dependencies among them, even with few steps, and they make the generative process differentiable, so gradients can steer generation toward a condition or reward. We first explain these advantages and then introduce the dynamics used throughout the survey.

1. This survey covers literature publicly available through November 2, 2026, including preprints.

2.1 Discrete Diffusion and Masked Generation

Consider a sequence $w = (w^1, \dots, w^L)$ over a vocabulary of V tokens. Each token can be represented by a one-hot vector $\boldsymbol{e}_k \in \mathbb{R}^V$. Probability distributions over the vocabulary form the simplex Δ^{V-1}: vectors with nonnegative entries that sum to one. A one-hot-encoded sequence is a vertex of the product $(\Delta^{V-1})^L$, whose V^L vertices represent all possible token sequences. We write $\boldsymbol{x}_t$ for the state of a generative process, with time running from a prior distribution p_0 at $t = 0$ to the data distribution p_1 at $t = 1$.

Discrete diffusion models generate text through jumps between token sequences. These jumps can be described by a continuous-time Markov chain, whose rates specify how likely each token changes [8, 9, 28]. In masked diffusion models (MDMs), generation starts with a fully masked sequence. A denoiser predicts a categorical distribution over tokens at each masked position, using all currently visible tokens as context. The sampler selects a set of masked positions to reveal and selects a token from their predicted marginals conditionally independently given the current sequence. With a standard sampler, revealed tokens then remain fixed, although corrector steps can re-mask or resample them with additional network evaluations [29]. Training uses a weighted cross-entropy loss to reconstruct masked tokens [3–5]. The denoiser can use context on either side of a masked position, and revealing several positions together enables parallel decoding.

2.2 Factorization Error in Parallel Decoding

A central obstacle to parallel decoding is that correct marginal predictions often do not produce the correct joint distribution. For a set of positions S, an explicit joint prediction would require probabilities for $V^{|S|}$ possible token combinations, which is intractable. The denoiser instead predicts one distribution per position, and the parallel sampler uses

$$\tilde{p}(w^S \mid \boldsymbol{x}_t) = \prod_{\ell \in S} p(w^\ell \mid \boldsymbol{x}_t). \tag{1}$$

Here, $w^S = (w^\ell)_{\ell \in S}$ denotes the tokens at the selected positions, p denotes the denoiser's predictions, and $\tilde{p}$ is the sampler's factorized approximation to their joint conditional distribution. The true conditional distribution usually does not factorize this way. For example, suppose the only possible completions are *she is* and *they are*, each with equal probability. The correct marginal predictions assign equal probability to *she* and *they* at the first position, and to *is* and *are* at the second. Sampling these positions independently produces all four combinations with equal probability, so half of the samples are invalid: *she are* or *they is*. The error comes from sampling independently, even though the marginal predictions are exact. Therefore, simply improving the denoiser's marginal predictions cannot resolve this error [15].

When each step reveals only a few tokens, newly sampled tokens become context for subsequent predictions,

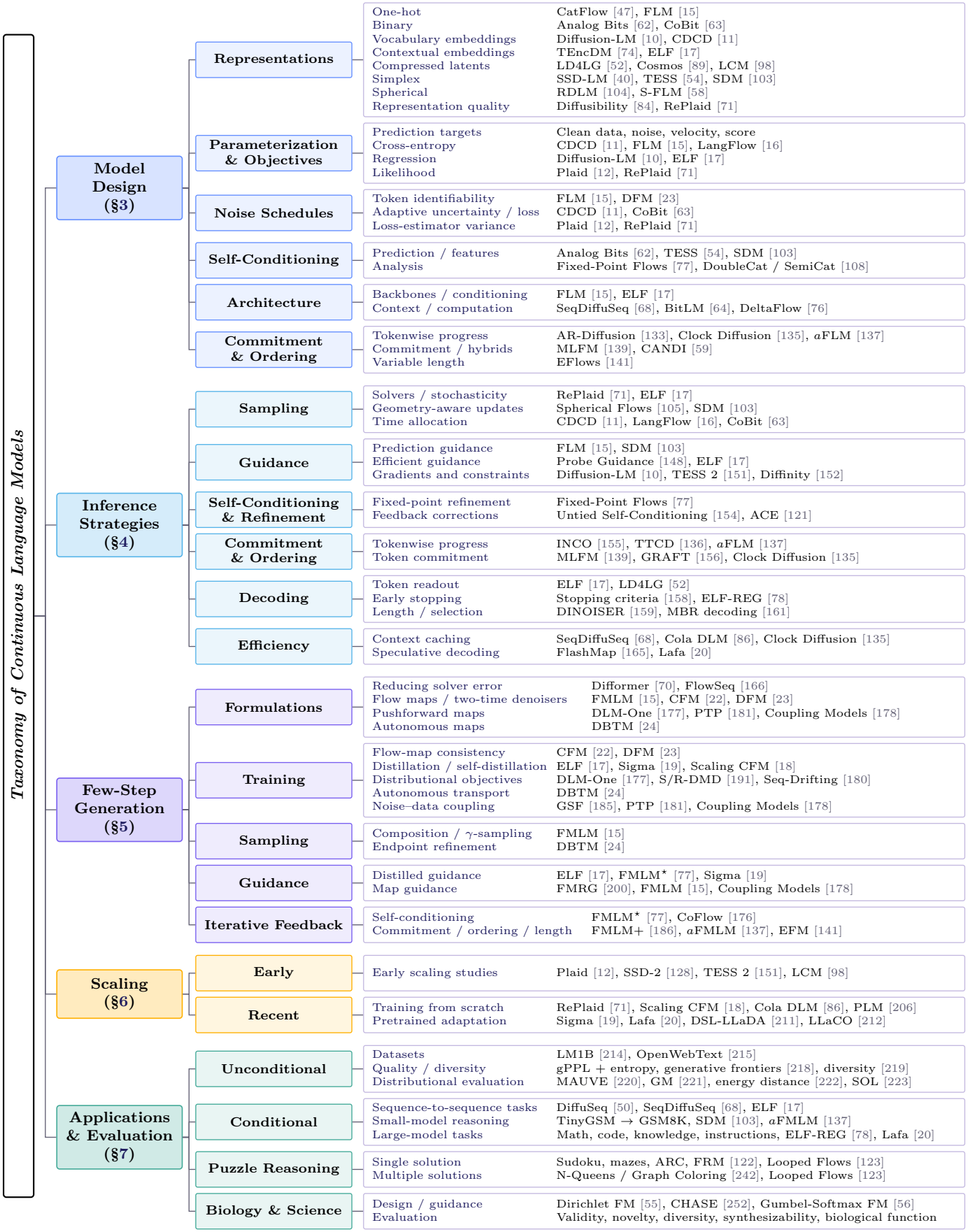


Fig. 2: A taxonomy of continuous language models, organized by the topics covered in Sections 3–7.

allowing the sampler to resolve these dependencies over many steps. Compressing generation into fewer steps removes those opportunities and forces more decisions to be made independently. This factorization error is therefore a central obstacle to few-step masked diffusion [30–32]. Methods that address this error introduce dependence between sampled positions, for example through a shared latent variable [32–34], a model that couples the token predictions [35, 36], a rectified source-target coupling [37], or a stochastic mask [38]. Continuous generators provide a direct way to represent this dependence: they jointly transform a noise sequence into token representations, rather than sampling tokens independently from per-position distributions.

2.3 Difficulty of Controllable Generation

Generation often needs to satisfy requirements beyond producing plausible text. A user may want a particular topic, sentiment, or property that can be scored by a reward model. A continuous state enables gradient-based control: when a classifier or reward model is differentiable with respect to the current representation, its gradient provides a direction for increasing the desired score. Classifier-free guidance provides another form of control by combining conditional and unconditional predictions [39]. Early continuous language models already used these ideas to steer text generation [10, 40].

A discrete state offers no such direction: a token cannot be nudged slightly toward a higher score. A gradient requires a continuous space in which small changes to the state are defined, and token identities alone provide no such space. Guidance must instead compare possible token replacements or modify their probabilities. Scoring many candidate changes is expensive, so practical methods approximate the guidance signal, adjust the denoiser's logits, or select among sampled candidates [7, 41, 42]. When guided tokens are sampled independently, however, the resulting update can still miss dependencies required by a sequence-level constraint. Continuous guidance can instead use gradients of a sequence-level objective to coordinate changes across positions in the jointly evolving state.

2.4 Continuous Processes for Discrete Data

Continuous dynamics provide a way to turn per-position predictions into joint generation. Rather than sampling tokens independently from these predictions, the sampler uses them to evolve a continuous representation of the entire sequence. An embedding denoiser predicts embedding vectors in parallel, and the sampler uses these predictions to refine the noisy sequence and rounds the final vectors to tokens [10]. Continuous models can also operate on one-hot representations or encoder latents.

Let $\boldsymbol{x}_1$ denote the continuous representation of a clean sequence and let $\boldsymbol{x}_0$ be independent Gaussian noise. This notation covers one-hot vectors, embeddings, and encoder latents. We reserve w^ℓ for the discrete token at position ℓ and $\boldsymbol{z}$ for self-conditioning inputs. Flow matching connects $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$ through an interpolant, often the straight path $I_t = (1-t)\boldsymbol{x}_0 + t\boldsymbol{x}_1$ [43–46]. This interpolant induces a path of marginal distributions $(p_t)_{t\in[0,1]}$, where $p_t = \mathrm{Law}(I_t)$. The optimal denoiser $D_t(\boldsymbol{x}) = \mathbb{E}[\boldsymbol{x}_1 \mid I_t = \boldsymbol{x}]$ predicts the expected clean representation given the current noisy state. For this straight interpolant, the corresponding ordinary differential equation (ODE) is

$$\dot{\boldsymbol{x}}_t = b_t(\boldsymbol{x}_t), \tag{2}$$

$$b_t(\boldsymbol{x}) = \mathbb{E}[\boldsymbol{x}_1 - \boldsymbol{x}_0 \mid I_t = \boldsymbol{x}] = \frac{D_t(\boldsymbol{x}) - \boldsymbol{x}}{1-t}, \tag{3}$$

where $\dot{\boldsymbol{x}}_t = \mathrm{d}\boldsymbol{x}_t/\mathrm{d}t$. That is, the velocity at $\boldsymbol{x}$ is the expected time derivative of the interpolant conditional on $I_t = \boldsymbol{x}$. A neural network learns the denoiser or velocity from noisy examples, and a numerical solver follows the resulting field from noise to data. Section 3 explains the representations, prediction targets, and training objectives used for language.

At each position, the denoiser predicts a mean conditioned on the noisy state of the entire sequence. Although these predictions are made positionwise, they jointly define a coupled velocity field. Variational flow matching [47] makes this distinction explicit: for this linear interpolant, even a factorized approximation to the endpoint posterior can recover the exact joint dynamics, provided it matches the per-position conditional means. Each mean depends on the entire noisy sequence, so factorization of the approximate posterior does not imply independence of the generated tokens. With an exact denoiser and exact integration, the resulting flow recovers the joint distribution of the continuous representations without explicitly representing probabilities over all V^L sequences. For tokenwise representations that uniquely identify each token, the final tokens can therefore be read out positionwise while preserving their dependencies.

This guarantee assumes exact integration, and taking only a few numerical steps can introduce substantial integration error. A *flow map* instead learns the finite-time transformation itself, so that one network evaluation can replace many solver steps. Because the map transforms shared noise into a whole sequence, one-step generation does not require a product approximation to the token distribution. This is the basis of the few-step methods reviewed in Section 5.

Continuous diffusion also permits stochastic sampling [48, 49]. For a probability path p_t generated by the velocity b_t, a corresponding stochastic differential equation (SDE) is

$$dX_t = b_t^\epsilon(X_t)\,dt + \sqrt{2\epsilon_t}\,dW_t, \tag{4}$$

where $b_t^\epsilon = b_t + \epsilon_t \nabla \log p_t$, W_t is Brownian motion, and $\epsilon_t \geq 0$ controls the noise injected during sampling. This SDE and the ODE in (2) have the same marginal distributions p_t [45]. Setting $\epsilon_t = 0$ gives deterministic sampling, whereas positive values give stochastic trajectories. For affine interpolants with Gaussian noise, such as the linear interpolant above, the score $\nabla \log p_t$ can also be obtained from the denoiser, as described in Section 3.2, so a trained model can support both types of samplers.

3 Continuous Language Models

The continuous framework introduced in Section 2 can be realized for language modeling through different model designs. The main design choices include the continuous representation, parameterization, training objective, noise schedule, self-conditioning mechanism, architecture, and generation order. In this section, we review these choices

Model	Representation	Formulation	Main loss	Few-step	Key feature
Diffusion-LM [10]	Learned vocab. emb.	Gaussian diff.	MSE + CE	–	Classifier guidance
DiffuSeq [50]	Learned vocab. emb.	Gaussian diff.	MSE + CE	–	Seq2seq denoising
CDCD [11]	Learned vocab. emb.	Gaussian diff.	CE	–	Learned time warp
Plaid [12]	Learned vocab. emb.	Gaussian diff.	ELBO	–	Learned noise schedule
Sigma [19]	Learned vocab. emb.	Gaussian diff.	ELBO + AR CE	Distill.	Uniformity regularization
SED [51]	Frozen vocab. emb.	Gaussian diff.	MSE + readout CE	–	Self-conditioning
LD4LG [52]	Compressed latents	Gaussian diff.	v-MSE	–	Pretrained AR decoder
LDLM [53]	Contextual latents	Gaussian diff.	MSE + recon. losses	–	Joint latent learning
SSD-LM [40]	Simplex	Gaussian logits	CE	–	Semi-AR blocks
TESS [54]	Simplex	Gaussian logits	CE	–	Self-conditioning
Dirichlet FM [55]	Simplex	Dirichlet flow	CE	Distill.	Classifier / CFG guidance
Gumbel-Softmax FM [56]	Simplex	Gumbel-softmax	CE	–	Straight-through guidance
Fisher-Flow [57]	Sphere	Geodesic flow	Riemannian FM	–	Fisher–Rao geometry
S-FLM [58]	Spherical emb.	Geodesic flow	CE	–	State dimension $d \ll V$
FLM [15]	One-hot	Linear flow	CE	–	Time reparameterization
LangFlow [16]	Learned vocab. emb.	VP flow	CE	–	Learned noise schedule
ELF [17]	Contextual emb.	Linear flow	Velocity MSE + CE	Distill.	Single-pass guidance
CFM [22]	One-hot	Flow map	CE + consistency	Self-dist.	Endpoint consistency
FMLM [15]	One-hot	Flow map	Semigroup CE	Distill.	FLM teacher
DFM [23]	One-hot	Flow map	CE/KL consistency	Self-dist.	Logit-space targets
DBTM [24]	One-hot	Endpoint map	Transport + CE	Direct	No time input
CANDI [59]	One-hot + mask	Gaussian + masking	Weighted CE	–	Decoupled corruption
CADD [60]	Tokens + emb.	Gaussian + masking	Masked CE	–	Latent-guided unmasking
CCDD [61]	Tokens + latent	Joint diffusion	Noise MSE + CE	–	Joint denoising

TABLE 1: Representative continuous language models, grouped into embedding and latent (purple), simplex and spherical (pink), flow and flow map (blue), and hybrid (teal) formulations.

and how they affect continuous language modeling, with representative models summarized in Table 1.

3.1 Continuous Representation of Language

The first decision is the continuous representation of language. Its dimensionality affects generation cost, its geometry shapes the generative trajectories, and its decoder determines how continuous predictions are mapped back to tokens. An overview is in Figure 3.

3.1.1 One-Hot and Binary Token Encodings

The simplest choice of text representation is *one-hot encoding*, where each token is given a coordinate vector $\boldsymbol{e}_k \in \mathbb{R}^V$. Continuous generation transports Gaussian noise toward these vectors using the interpolant $I_t = (1-t)\boldsymbol{x}_0 + t\boldsymbol{x}_1$ introduced in Section 2. The evolving state lies in the ambient space $\mathbb{R}^V$, while the optimal denoiser lies on the probability simplex [11], since

$$D_t(\boldsymbol{x})^\ell = \sum_{k=1}^{V} \mathbb{P}(\boldsymbol{x}_1^\ell = \boldsymbol{e}_k \mid I_t = \boldsymbol{x})\boldsymbol{e}_k \quad \in \quad \Delta^{V-1}. \tag{5}$$

Its coordinates are therefore token probabilities, which can be parameterized by a softmax and learned with cross-entropy, as in **CatFlow** [47] and **Flow Language Models** (FLM) [15]. The representation requires no learned encoder or decoder, with the final prediction decoded by argmax. All one-hot vectors are equally separated, so the encoding imposes no prior notion of token similarity.

The main cost is dimensionality. A one-hot sequence occupies $\mathbb{R}^{L\times V}$, making the state and dense input/output projections expensive for large vocabularies. *Binary encodings* provide a more efficient representation in this regard, as they can distinguish V symbols using only $\lceil \log_2 V \rceil$ bits. **Analog Bits** [62] introduced diffusion over such codes by treating their bits as real-valued coordinates during generation and thresholding the final predictions back to bits. This construction supports ordinary continuous diffusion without learning an embedding space, and was demonstrated on image generation and image captioning.

CoBit [63] develops bitstream diffusion for language, grouping bits into transformer inputs and replacing the vocabulary-sized prediction head with a compact bitwise head. It also exploits the closed-form posterior of an isolated noisy bit, learning a contextual correction that accounts for dependencies among bits and tokens. **BitLM** [64] uses binary token codes in a lightweight bitwise head that generates several tokens jointly within each block, while retaining causal attention across blocks.

Unlike one-hot vectors, binary codes have unequal pairwise distances, determined by the number of differing bits. These distances depend on the code assignment and need not reflect linguistic similarity. **H-CoBit** [65] adds linguistic structure by augmenting token codes with codes for clusters of pretrained token embeddings and denoising both jointly.

3.1.2 Vocabulary Embeddings

Vocabulary embeddings allow the geometry of token representations to be learned. An embedding table $E \in \mathbb{R}^{d\times V}$

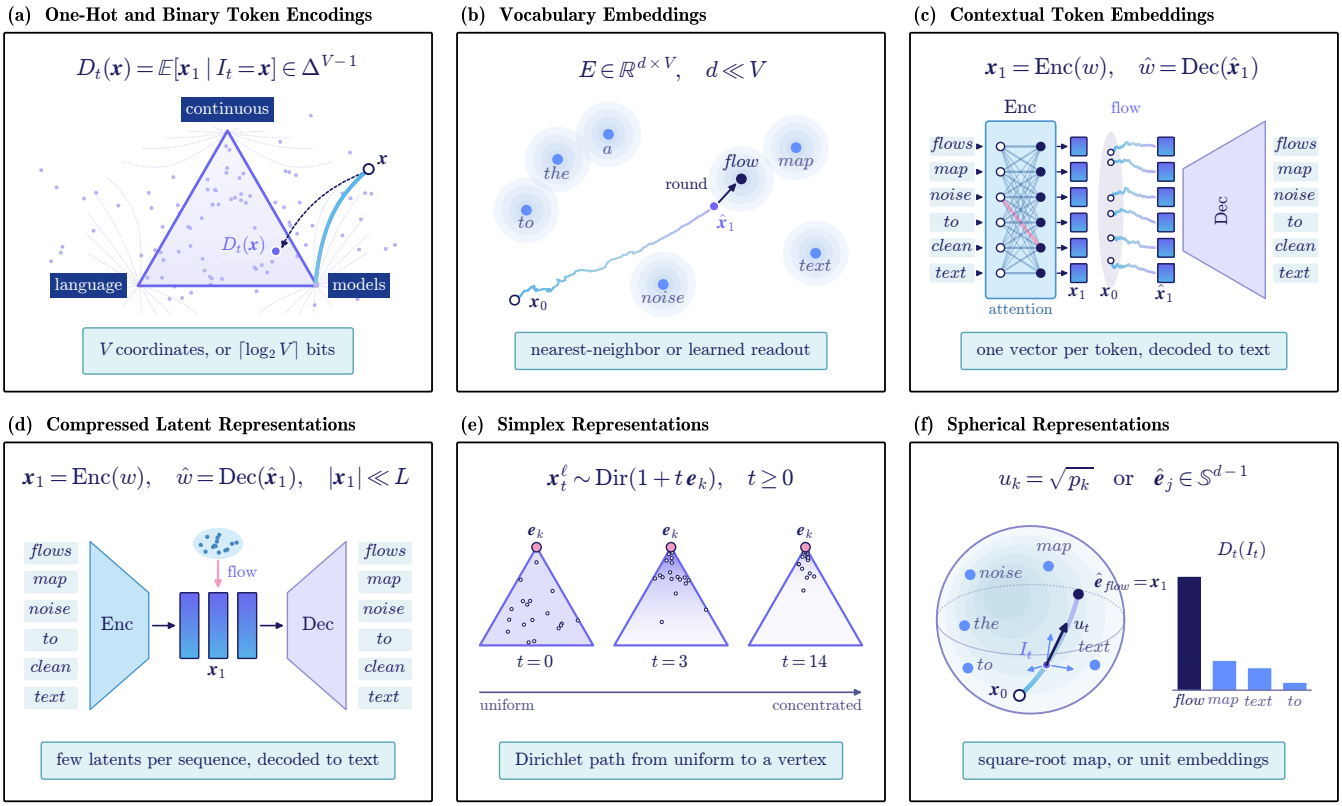


Fig. 3: Continuous representations of language for continuous language models. (a) One-hot flows carry Gaussian noise toward the vertices of the simplex, and the denoiser output $D_t(\boldsymbol{x})$ lies on the simplex. Binary codes reduce the V coordinates to $\lceil \log_2 V \rceil$ bits. **(b)** Vocabulary embedding models run the flow over a table of token embeddings and map the final state $\hat{\boldsymbol{x}}_1$ to a token by nearest-neighbor rounding or a learned readout. **(c)** Contextual embedding models generate encoder outputs, one vector per token, and use a decoder to recover the text. **(d)** Latent models compress a sequence into a few vectors, run the flow on those vectors, and decode the result to text. **(e)** In simplex models, each state is a distribution over the vocabulary. The Dirichlet path starts uniform and concentrates at the target vertex $\boldsymbol{e}_k$ as $t \geq 0$ grows. **(f)** Spherical models either map distributions to the sphere with a square root or use unit-norm token embeddings. In the embedding case shown, the denoiser probabilities $D_t(I_t)$ weight the directions toward each embedding to give the velocity u_t.

assigns each token a vector, ideally with $d \ll V$, and generated vectors are mapped back to tokens by nearest-neighbor rounding or a learned readout. The table can be pretrained or learned jointly with the generator. Pretrained embeddings provide an existing geometry, but it may be poorly suited to flows. Joint learning allows the representation to adapt, but can also reduce the training loss by collapsing token vectors together or increasing their scale relative to the noise. Methods therefore differ in how they avoid these failure modes during training.

SED [51] uses normalized vocabulary embeddings from a separately pretrained BERT model [66] and freezes them after finding joint learning unstable. Freezing avoids changes to the representation during diffusion training, but also prevents it from adapting to the generative task. **Smoothie** [67] likewise freezes BERT embeddings, but it adds noise to the distances between each position and all token embeddings, so that corruption first blurs semantically similar tokens. **Lafa** [20] takes an intermediate approach by adapting a pretrained AR model's embedding table while preserving each vector's original norm. Its directions remain trainable, allowing the geometry to change without making denoising artificially easier through norm growth.

For embeddings learned jointly from scratch, an early approach is to combine denoising with token reconstruction. **Diffusion-LM** [10] uses a reconstruction loss to keep token embeddings decodable and a weighted L2 penalty to discourage excessive embedding norms. Related objectives are used by **DiffuSeq**, **SeqDiffuSeq**, and **GENIE** [50, 68, 69]. **Difformer** [70] finds that reconstructing clean embeddings can become too easy to provide sufficient regularization. Its anchor loss instead requires the denoised predictions to decode to the correct tokens, providing stronger supervision for both the embeddings and the denoiser.

Another approach learns the token embeddings mainly through denoising. **CDCD** [11] replaces embedding regression with cross-entropy and normalizes the embeddings to prevent unbounded norm growth. **LangFlow** [16] likewise combines cross-entropy with normalized embeddings. **Plaid** and **RePlaid** [12, 71] instead learn normalized embeddings through a complete likelihood objective, balancing reconstruction and diffusion terms rather than optimizing denoising error alone (Section 3.2).

Controlling norms does not fully determine the geometry. **Sigma** [19] finds that learned unit-norm vocabulary vectors concentrate in a few effective dimensions. It introduces

a regularizer that encourages a more uniform distribution over the hypersphere, improving few-step generation. The constraint applies to the data embeddings, while Gaussian diffusion evolves in the surrounding Euclidean space.

3.1.3 Contextual Token Embeddings

Contextual embeddings represent a token together with its role in the sequence [72, 73]. By distributing information about words and their relationships across positions, they give the denoiser a target space in which linguistic structure is already expressed. These representations are typically obtained from a transformer, whose attention lets each token vector incorporate the surrounding text or its prefix. Comparisons with vocabulary embeddings support the benefits of contextualization for continuous generation [17, 53, 74]. Contextualization and sequence compression are partly independent design choices.

One approach reuses the hidden states of a pretrained language model. Early work by Lovelace et al. [52] applied diffusion to frozen BART encoder outputs, compressed into a fixed number of latent vectors. **TEncDM** [74] generates frozen encoder outputs from models such as BERT and T5 [75], while **Embedded Language Flows** (ELF) [17] use T5 representations as the targets of flow matching. **DeltaFlow** [76] adopts the same representation with a different denoising architecture. Context need not span the full sequence. **FMLM*** and **ELF-REG** use causal hidden states from GPT-2 and Qwen, respectively [77, 78], while **MF-1** [79] applies T5 independently within short token blocks. Contextual representations can also be pretrained through autoencoding, as in **TextLDM** and **AURORA-LM** [80, 81].

Decoding these representations requires accounting for the errors left by generation. Lovelace et al. [52] use BART's pretrained AR decoder, while **CoDAR** [82] trains a separate AR decoder on perturbed outputs of a frozen Qwen3-Embedding encoder [83]. Both condition each token on the generated representations and previously decoded text. Other methods instead perform parallel decoding. TEncDM and MF-1 train separate bidirectional decoders on corrupted representations, while TextLDM and AURORA-LM use the decoders learned during autoencoding. ELF uses the denoising model itself to predict tokens in a final decoding mode.

A representation that reconstructs text well need not be easy to generate. Zhang et al. [84] find that T5Gemma-2 [85] encodings overly separate even plausible alternative words, which they attribute to training against one-hot token labels. Distilling the encoder with soft token probabilities brings their representations closer and improves generation in the resulting frozen space. **LDLM** [53] adapts pretrained representations through joint training, using a trainable encoder to reshape frozen GPT-2 states. It gradually introduces diffusion gradients into the encoder and trains the decoder on perturbed latents, allowing the representation to adapt while preserving recoverability. **Cola DLM** [86] first pretrains a causal VAE [87] with additional masking losses, then trains it jointly with the flow model, using a reference encoder to regularize changes to the latent space.

3.1.4 Compressed Latent Representations

Even low-dimensional token representations require the generator to process every token position at each step. Compressing the sequence into a compact *latent space* reduces this repeated computation. It can also potentially make learning the generator easier, as a short latent sequence can describe information shared across a sentence or span, leaving its detailed wording to the decoder.

One approach learns compressed representations for reconstructing the original text. **LD4LG** [52] uses a fixed number of learned queries to cross-attend to frozen BART or T5 encoder outputs. Diffusion generates the resulting latent vectors, which the pretrained AR decoder maps to variable-length text. **PLANNER** [88] selects a small set of contextual BERT states to represent a paragraph and trains a GPT-2 decoder to reconstruct it. **Cosmos** [89] also compresses contextual states with learned queries [90], but reconstructs tokens in parallel. It trains its autoencoder with masking, noise, and latent dropout to improve robustness to imperfect latents. **JPEG-DLM** [91] jointly trains a compressor over frozen T5 states, a flow model, and a parallel decoder. It learns to predict compressed representations of denoised text from corrupted views, using a slowly updated copy of the compressor to provide targets. Feature and token reconstruction losses keep the predicted representations decodable.

Other methods organize the compressed representation around local spans of text. **Manta-LM** [92] downsamples the sequence with a convolutional VAE, while **CALM** [93] represents fixed-size token chunks as individual continuous vectors and generates these vectors autoregressively. **Cola2** [94] learns segmentation into variable-length spans, allowing the number of tokens represented by each latent to depend on the text. **BLD** [95] compresses contiguous spans into vectors and pairs them with AR decoding branches, allowing several spans to be expanded into text in parallel. **SLD** [96] similarly decodes one latent per segment of a long output with an AR decoder.

Some approaches focus on using compressed latents to specify semantic content while leaving variation in wording to a generative decoder. **DGLM** [97] generates a single continuation embedding in the space of the contrastively trained Sentence-T5 encoder, then conditions an AR model on it. **Large Concept Model** (LCM) [98] instead generates sentence embeddings in the space of SONAR [99], conditioning each on the previous ones, and decodes them with its frozen SONAR decoder. **PSF** [100] uses a flow to model states computed from the context preceding each block. These states guide parallel generation across blocks, with autoregressive decoding within each block. **DiLaDiff** [33] conditions a masked diffusion decoder on generated continuous latents, using the shared latent information to coordinate its token predictions, and **CRoCoDiL** [101] obtains such a decoder by fine-tuning a pretrained masked diffusion model jointly with the encoder.

3.1.5 Simplex Representations

Simplex representations describe each position by nonnegative vocabulary weights that sum to one. Generation continuously refines these weights, with each coordinate interpreted as the probability assigned to a vocabulary item. This interface also supports guidance from existing text classifiers without retraining them for the diffusion process [40].

An early approach obtains these probabilities through a softmax applied to evolving logits. **SSD-LM** [40] represents data tokens by logit vectors whose softmax is approximately one-hot and adds Gaussian noise to these logits, with the denoiser receiving their softmax as input. **TESS** [54] uses the same approach and feeds back its previous vocabulary prediction for self-conditioning (Section 3.4). In both cases, diffusion takes place in unconstrained logit space. A related idea appears in **Duo** [31], which applies tempered softmax on Gaussian diffusion to warm-start the training of uniform discrete diffusion.

Other approaches define updates directly on the simplex, with early approaches including Bayesian probabilistic updates in **BFN** [102]. For flow-based models, the choice of probability path needs to be adapted to the simplex. **Dirichlet Flow Matching** [55] shows that straight-line interpolation from simplex noise to token vertices produces shrinking conditional supports and discontinuous velocity fields. It instead uses Dirichlet paths that retain overlapping support at intermediate times while gradually concentrating near the target vertices. **Simplex Diffusion Models** (SDM) [103] separately control the mean and concentration of their Dirichlet paths and derive closed-form reverse transitions, allowing sampling with adjustable stochasticity without integrating an ODE. **Gumbel-Softmax Flow Matching** [56] uses another family of paths, sharpening initially diffuse token distributions as the temperature decreases.

3.1.6 Spherical Representations

Spherical representations evolve token states along a hypersphere, where proximity is measured by angular distance. This geometry connects naturally to both categorical distributions and learned embeddings. The former can be mapped from the simplex to a sphere through a square-root transformation, while the latter uses normalized vectors whose angular distances correspond to cosine similarities. In the latter, learned embeddings can occupy a compact space whose dimension is independent of vocabulary size.

For categorical distributions, the elementwise transformation $u_k = \sqrt{p_k}$ maps the simplex to the positive orthant of the unit sphere. Under this map, the Fisher–Rao metric corresponds to the spherical metric up to a constant scale. **Fisher-Flow** [57] uses this connection to train with Riemannian flow matching along spherical geodesics. **RDLM** [104] constructs stochastic bridges toward token vertices on the hypersphere, combining their drifts according to predicted token probabilities. To handle large vocabularies, RDLM represents each token index as digits in a chosen base and evolves these digits on a product of smaller spheres. This reduces the state dimension and makes the bridges converge more gradually toward their endpoints.

The second construction learns unit-norm token embeddings in a space of dimension $d \ll V$. **S-FLM** [58] trains on geodesic interpolations between uniform spherical noise and clean embeddings. Its denoiser predicts token probabilities with a cross-entropy objective, and these probabilities weight the tangent directions toward candidate embeddings to define the velocity field. **Spherical Flows** [105] uses von Mises–Fisher paths, which progress from uniform noise to increasingly concentrated distributions around each target embedding. These paths admit tractable expressions for both velocity and score. Both can therefore be obtained from a single model's token predictions, supporting deterministic sampling and stochastic refinement on the sphere. **SBD** [106] uses Brownian motion on the sphere to corrupt token embeddings. Its heat-kernel transitions admit ancestral bridges, supporting reverse updates conditioned on both the current noisy state and a predicted clean token.

3.1.7 What Makes a Good Representation?

The discussion so far raises a natural question of what makes a representation effective for continuous generation. A general answer remains open, but existing empirical studies offer guidance for choosing and learning representations.

Conventional measures of representation quality do not fully predict generation quality. Zhang et al. [84] find that improvements in pretrained encoders can benefit diffusion, but strong discriminative performance alone does not ensure a suitable generation space. Distilling soft token predictions brings plausible alternatives closer in representation space and improves generation. Du and Ma [107] find that accurate reconstruction of clean representations and low denoising error can coexist with unreliable decoding of generated states. Even small representation errors can change the decoded tokens when the decoder is sensitive to perturbations. These observations motivate evaluating the representation and decoder together on the imperfect states encountered during generation.

Representation choices also interact with the training objective, noise schedule, and self-conditioning, described in the later sections. In RePlaid, Yang et al. [71] find that adding an auxiliary cross-entropy loss disperses low-rank embeddings and worsens its likelihood bound, illustrating that low effective dimension is not necessarily a failure. For one-hot encodings, Pynadath et al. [59] and Lee et al. [15] show how vocabulary size affects token identifiability under Gaussian corruption, with the latter allocating training times to the narrow interval where one-hot tokens become distinguishable. **DoubleCat** and **SemiCat** [108] further connect self-conditioning to representation geometry, finding that it helps identify tokens in noise-dominated states and reduces the mismatch between training near clean token embeddings and generation through uncertain intermediate predictions. Establishing such understanding across broad choices of representations and comparing between them remains an important future direction.

3.2 Parameterization and Objectives

With a representation in place, the next design choices are the prediction targets of the network and the training loss. Although different choices can yield equivalent models at the optimum, they can substantially affect learning in practice.

3.2.1 Prediction Parameterizations

For the interpolants with Gaussian noise introduced in Section 2, the velocity or score needed for generation can be learned directly or obtained from a network that predicts the clean representation or the added noise. For the linear interpolant $I_t = (1-t)\boldsymbol{x}_0 + t\boldsymbol{x}_1$ with independent Gaussian

noise $\boldsymbol{x}_0 \sim \mathcal{N}(0, \sigma^2 I)$, these quantities are related through the denoiser $D_t(\boldsymbol{x}) = \mathbb{E}[\boldsymbol{x}_1 \mid I_t = \boldsymbol{x}]$. For $0 < t < 1$,

$$\begin{aligned} \mathbb{E}[\boldsymbol{x}_0 \mid I_t = \boldsymbol{x}] &= \frac{\boldsymbol{x} - tD_t(\boldsymbol{x})}{1-t}, \\ b_t(\boldsymbol{x}) &= \frac{D_t(\boldsymbol{x}) - \boldsymbol{x}}{1-t}, \\ \nabla \log p_t(\boldsymbol{x}) &= \frac{tD_t(\boldsymbol{x}) - \boldsymbol{x}}{(1-t)^2\sigma^2}. \end{aligned} \tag{6}$$

A learned prediction of any one of these quantities can therefore be converted into the others, including the velocity or score required by the sampler.

Among those, clean-data prediction is common in continuous language models [10, 11]. One reason is that directly predicting noise or velocity requires representing a Gaussian component across the full ambient space. In **FLM**, Lee et al. [15] find that this makes velocity prediction difficult in one-hot spaces where the vocabulary dimension greatly exceeds the network's hidden width. Predicting the clean representation instead allows the noisy input to be incorporated into the velocity through the analytic conversion above. In **ELF**, Hu et al. [17] likewise find clean-embedding prediction more reliable than noise or velocity prediction across contextual representations.

The quantity predicted by the network need not be the quantity compared in the loss. ELF, for example, predicts clean embeddings but trains with a squared error on the corresponding velocity.

3.2.2 Cross-Entropy Objectives

A simple objective for learning the denoiser is to regress the clean representation with squared error. For one-hot representations, the network can instead predict a softmax distribution, $q_t^\theta(\boldsymbol{x})^\ell \in \Delta^{V-1}$, and minimize cross-entropy,

$$\mathcal{L}_{\mathrm{CE}}(\theta) = \mathbb{E}_{t,\boldsymbol{x}_0,w}\left[-\sum_{\ell=1}^{L} \log q_t^\theta(I_t)^\ell_{w^\ell}\right]. \tag{7}$$

Cross-entropy is a strictly proper scoring rule, meaning that its expected loss is minimized by the true token probabilities conditional on the noisy sequence [109]. For one-hot representations, these probabilities are also the expected data vector $D_t(\boldsymbol{x})^\ell$. **FLM** [15] thus learns its denoiser $D^\theta = q^\theta$ with cross-entropy and obtains the velocity through Equation (6).

The same idea applies to vocabulary embeddings. Given an embedding table E, the predicted token probabilities yield an estimate of the expected data embedding through

$$D_t^\theta(\boldsymbol{x})^\ell = \sum_{k=1}^{V} q_t^\theta(\boldsymbol{x})^\ell_k \, E\boldsymbol{e}_k. \tag{8}$$

CDCD [11] uses this expected embedding to obtain the score through the Gaussian conversion described above, an approach it calls *score interpolation*. Cross-entropy can thus train a continuous denoiser even when its state consists of compact embeddings instead of vocabulary-sized vectors. **LangFlow** [16] takes the same approach and uses cross-entropy for flow matching with vocabulary embeddings. Although squared error on the predicted probabilities has the same optimum [110], its gradients behave differently. With softmax outputs, the cross-entropy gradient with respect to the logits is simply the predicted probabilities minus the one-hot target. Squared error additionally multiplies this difference by the softmax Jacobian, which can make gradients very small when the model is confidently wrong [111]. **SERF** [112] further adds a self-distillation loss whose teacher is an exponential moving average of the model given a less noisy input, to speed up training.

3.2.3 Regression Objectives

Although recent methods based on vocabulary embeddings often use cross-entropy for training, regression remains useful, and it becomes a natural choice for contextual representations and compressed latents.

Early models, such as **Diffusion-LM** [10], use squared-error losses to predict the clean representation over learned vocabulary embeddings, with added regularization to prevent collapse (Section 3.1.2). **ConvergeFlow** [113] predicts a weighted average of vocabulary embeddings (8) like CDCD and LangFlow, but trains this continuous prediction with squared error. Its weights need not recover data probabilities, since different weights can yield the same mean embedding when $V > d + 1$. Starting from a pretrained LangFlow with its embedding table frozen, continued regression training gives competitive generation. **Lafa** [20] combines cross-entropy on token probabilities with squared error on the corresponding mean embeddings. Its ablations find that the two losses together improve generation over either alone. These results show that regression can play a complementary role even for models trainable with cross-entropy.

For contextual or compressed representations, per-token probabilities generally do not determine the expected clean representation, so the weighted-sum construction in Equation (8) no longer applies directly. Regression instead learns this continuous target. Writing $\boldsymbol{x}_1$ for the clean representation and $\boldsymbol{x}_t$ for its noisy counterpart, a common objective is

$$\mathcal{L}_{\mathrm{reg}}(\theta) = \mathbb{E}_{t,\boldsymbol{x}_0,\boldsymbol{x}_1}\left[\lambda(t)\left\|D_t^\theta(\boldsymbol{x}_t) - \boldsymbol{x}_1\right\|_F^2\right], \tag{9}$$

where $\lambda(t)$ controls the weight assigned to each noise level. A decoder then maps the generated representations to text. **TEncDM** [74] pairs squared-error prediction of frozen encoder states with a separate bidirectional decoder trained by cross-entropy on perturbed states. **Cosmos** [89] similarly separates latent regression from token reconstruction after learning a compressed autoencoding space.

ELF [17] shares a network between denoising and decoding. During denoising, it predicts the contextual representation of clean data and uses $\lambda(t) = (1-t)^{-2}$, corresponding to squared error on the velocity. In decoding mode, the same network processes perturbed embeddings and predicts tokens through a learned output projection, with cross-entropy supervision. A mode indicator on the input distinguishes the two tasks, allowing continuous denoising and token decoding to benefit from shared parameters.

3.2.4 Likelihood Objectives

Many generative models, including autoregressive language models, are trained by maximizing data likelihood. This fits the model distribution to observed data and provides a

common criterion for evaluating held-out text. Continuous language models can follow the same principle, but their likelihood must account for the mapping from continuous representations to categorical tokens. The relevant quantity is the probability of a token sequence,

$$p^\theta(w) = \int p^\theta_{\text{dec}}(w \mid \boldsymbol{x})\, p^\theta_{\text{cont}}(\boldsymbol{x})\, d\boldsymbol{x}, \tag{10}$$

where p^θ_{cont} describes the continuous representation given to the decoder p^θ_{dec}. A continuous density evaluated at an embedding is not itself a text probability. The integral collects probability mass through the decoder, yielding a distribution over text.

Computing this integral is generally intractable. **Plaid** [12] adopts *variational diffusion* [114] to optimize a tractable likelihood bound. Let $\boldsymbol{x}_1$ denote the data token embeddings where $\boldsymbol{x}_1^\ell = E\boldsymbol{e}_{w^\ell}$ and define the Gaussian interpolant $I_t = \alpha_t \boldsymbol{x}_0 + \beta_t \boldsymbol{x}_1$, where $\boldsymbol{x}_0 \sim \mathcal{N}(0, I)$. With $\text{SNR}(t) = \beta_t^2/\alpha_t^2$ increasing in t, the bound takes the form

$$\begin{aligned} -\log p^\theta(w) \le\ & \mathcal{L}_{\text{dec}}(\theta, w) + \mathcal{L}_{\text{prior}}(\alpha_0, \beta_0, w) \\ & + \frac{1}{2}\mathbb{E}_{t,\boldsymbol{x}_0}\left[\text{SNR}'(t)\left\|D^\theta_t(I_t) - \boldsymbol{x}_1\right\|_F^2\right], \end{aligned} \tag{11}$$

where $t \sim \mathcal{U}[0,1]$ and with the decoding and prior terms

$$\begin{aligned} \mathcal{L}_{\text{dec}}(\theta, w) &= \mathbb{E}_{\boldsymbol{x}_0}\left[-\sum_{\ell=1}^{L} \log p^\theta_{\text{dec}}(w^\ell \mid I_1)\right], \\ \mathcal{L}_{\text{prior}}(\alpha_0, \beta_0, w) &= \text{KL}\big(\mathcal{N}\big(\beta_0 \boldsymbol{x}_1, \alpha_0^2 I\big) \,\big\|\, p_{\text{prior}}\big), \end{aligned} \tag{12}$$

where p_{prior} is the Gaussian distribution used to initialize generation. The decoding term trains a tokenwise decoder on the least-noisy state I_1, and the prior term aligns the distribution of the most-noisy state I_0 with the sampling noise. The remaining term is embedding regression, with weights determined by the noise schedule. It can be estimated by corrupting an example at a sampled time, without simulating a generation trajectory. For deterministic encoder representations, an analogous likelihood bound can be constructed using a specified Gaussian corruption process and decoder. For stochastic latent encoders, the variational objective additionally includes the corresponding posterior and KL terms [115].

The bound provides a single objective for jointly learning the embeddings, denoiser, and decoder. Collapsing embeddings may reduce denoising error, but also makes tokens harder to reconstruct, which the full objective accounts for. **RePlaid** [71] revisits this approach with modern transformer backbones and shows competitive likelihoods at scale.

Likelihood bounds can also evaluate models trained with other objectives. **LangFlow** [16] trains with denoising cross-entropy but derives an ODE-based bound for likelihood evaluation. Taking the exponential of these negative log-likelihood bounds after normalization by sequence length gives upper bounds on perplexity.

3.3 Noise Scheduling and Time Reparameterization

In continuous diffusion and flow models, a denoiser must learn across corruption levels whose difficulty and information content can vary substantially. Allocating computation across these levels can improve both training and generation. For a fixed Gaussian corruption path and noise-level range, changing the time parameterization or time-sampling distribution can be expressed as changing the loss weighting across noise levels [11, 114]. Many of these choices are also used to place generation steps, which we discuss in Section 4.1.

In some cases, the representation and corruption process provide an analytical basis for choosing the schedule. For one-hot representations under a linear Gaussian interpolant, for instance, the probability of recovering the clean token by argmax changes sharply within a short time interval, especially for large vocabularies [59]. Uniform time allocation therefore assigns relatively few training examples to the interval where token identity becomes distinguishable. **FLM** [15] retains the linear interpolant and samples times through a *time reparameterization* in which the recovery probability increases linearly. Potaptchik et al. [23] implement the same principle through the noise schedule, choosing the interpolant coefficients so that the recovery probability increases approximately linearly with time.

Schedules can also be chosen adaptively from losses observed during training. **SeqDiffuSeq** [68] adjusts a separate noise schedule for each token position so that its denoising loss changes approximately linearly with time. **CDCD** [11] uses cross-entropy as a proxy for remaining token uncertainty and learns a *time warp* that makes this quantity approximately linear. This allocates more training samples to noise levels where uncertainty changes rapidly. **LangFlow** [16] follows a similar principle, fitting the uncertainty curve over the log noise-to-signal ratio. **CoBit** [63] applies the same principle to bitstreams through the information-guided allocation of InfoNoise [116], which estimates how fast this uncertainty changes from denoising errors. **Lafa** [20] learns a separate time reparameterization for each token position, using denoising cross-entropy to allocate more training samples to noise levels with higher losses.

Likelihood training provides another criterion for choosing the schedule. **Plaid** and **RePlaid** [12, 71] learn how often to sample each noise level to reduce the variance of the training loss estimate. For a fixed range of noise levels, the weights in the likelihood bound compensate for changes in sampling frequency, leaving the expected objective unchanged. RePlaid also finds that the resulting schedule makes token cross-entropy decrease approximately linearly from noise to data, connecting to the aforementioned uncertainty-based schedules, even though the schedule is learned without explicitly targeting this behavior.

3.4 Self-Conditioning

Self-conditioning is a simple and effective technique introduced in **Analog Bits** [62] and subsequently adopted broadly for continuous text generation [11, 51]. At each generation step, the denoiser receives its previous denoising estimate as an additional input, allowing it to refine an existing prediction. A common training scheme computes a preliminary prediction without gradients for half of the

examples and feeds it back for the supervised prediction, and the remaining examples use a null conditioning input,

$$\hat{\boldsymbol{x}}_1 = D_t^\theta(\boldsymbol{x}_t, \boldsymbol{z}), \quad \boldsymbol{z} = \begin{cases} \mathrm{sg}\big(D_t^\theta(\boldsymbol{x}_t, \varnothing)\big) & \text{with probability } \frac{1}{2}, \\ \varnothing & \text{otherwise.} \end{cases}$$

Here, sg denotes the stop-gradient operation. This adds modest training overhead, while generation reuses the prediction already computed at the preceding step and requires no additional denoiser evaluation.

Methods differ in what they feed back. **SED**, **SeqDiffuSeq**, and **ELF** use denoised token embeddings [17, 51, 68]. **TESS** [54] feeds back the previous token distribution, averaging it with the distribution obtained from the current noisy logits. **SDM** [103] leverages hidden features from the preceding network evaluation, allowing feedback to retain information beyond the clean prediction [117].

The empirical success of self-conditioning has motivated several explanations. **Plaid** and **Fixed-Point Flows** interpret self-conditioning through fixed-point refinement [12, 77]. Repeatedly feeding predictions back at a fixed noisy state and time can refine the denoising estimate. Ordinary self-conditioned sampling performs one such iteration per flow step, initialized with the preceding prediction. **DoubleCat** and **SemiCat** [108] show that feedback makes token information easier to identify under strong noise and helps bridge the gap between training near clean token representations and generating through uncertain intermediate predictions. **TREC** [118] and **FastDiSS** [119] instead modify training, the former keeping self-conditioning beneficial through reinforcement learning and the latter perturbing the self-conditioning input to match the errors of few-step sampling. Related work also finds that self-conditioning stabilizes predictions in regions poorly covered during training [120]. Repeated feedback can also increase confidence and reinforce repetitive text [74, 121], motivating the inference adjustments discussed in Section 4.3.

Recent methods extend this feedback into more general recurrence. **FRM** [122] uses predictions from its own inference trajectories as conditioning, while **Looped Flows** [123] feeds back hidden states, trained across progressively lower corruption levels with shared noise. Both learn recurrence through denoising losses without backpropagating through the full trajectory. During generation, recurrence preserves useful computation while the flow progressively refines the output, thereby extending the approach of self-conditioning.

3.5 Architecture

The denoising network must combine information across positions while accounting for their noise level. Bidirectional transformers are the most common architectural choice, as in **CDCD** and **RDLM** [11, 104]. **FLM** [15] adopts the *diffusion transformer (DiT)* [124] backbone used in recent discrete diffusion models, with rotary position embeddings [125], while **RePlaid** [71] modernizes Plaid with adaptive normalization [126] and learned residual gates.

The noise level can condition the network in several ways. **SED** [51] adds a time embedding to the projected input, whereas CDCD uses time-dependent shifts and scales of normalized features within each layer. DiT backbones similarly use adaptive layer normalization for time conditioning. **ELF** [17] instead prepends *control tokens* encoding time, guidance strength, and whether the network is denoising or decoding. Replacing adaptive normalization with these tokens slightly improves generation quality while reducing the parameter count in its experiments. **LangDiT** [127] revisits normalization, noise conditioning, and self-conditioning together, and improves generation across model formulations.

Self-conditioning introduces another input. ELF concatenates the noisy representation and the previous denoising prediction along the feature dimension, then projects them back to the embedding width. **LangFlow** [16] uses a residual form of this fusion, adding learned projections of both inputs to the original noisy representation. These additional projections are initialized to zero, so the network initially processes the unmodified noisy input.

For conditional generation, the architecture determines how observed context interacts with the evolving sequence. **DiffuSeq** processes the source and noisy target jointly, whereas **SeqDiffuSeq** encodes the source once and reuses its outputs throughout denoising [50, 68]. **SSD-2** [128] uses causal attention for the clean prefix and bidirectional attention within the block being generated, allowing multiple blocks to be trained in parallel against their preceding context. Recent large models, including **Sigma** [19] and **Lafa** [20], adopt this attention pattern for prefix caching. Because the prefix does not attend to the noisy block, its keys and values can be cached across denoising steps. **Lafa** [20] also uses tokenwise time conditioning, which keeps the time input of the prefix fixed.

Some other architectures focus on reducing the computation repeated during generation. **BitLM** [64] separates a large context-processing backbone from a lightweight diffusion head. The backbone computes context once per block, while the head repeatedly refines the next block under that fixed conditioning. **DeltaFlow** [76] instead reduces the cost of mixing information across positions. It replaces most self-attention layers with gated delta networks [129, 130] that scan the sequence in both directions, with memory updates conditioned on the noise level.

3.6 Commitment and Ordering

Most continuous flows denoise all positions together using a shared time variable. Resolving some parts earlier, however, can provide context that makes the remaining predictions easier [131, 132]. Continuous models can exploit this ordering by letting positions advance at different rates or by committing selected predictions as fixed context.

Giving each position its own time allows some tokens to become informative while others are uncertain [133, 134]. This gives an *asynchronous* version of the linear interpolant,

$$I_{\boldsymbol{\tau}}^\ell = (1 - \boldsymbol{\tau}^\ell)\boldsymbol{x}_0^\ell + \boldsymbol{\tau}^\ell \boldsymbol{x}_1^\ell, \qquad \boldsymbol{\tau} \in [0,1]^L, \tag{13}$$

where $\boldsymbol{\tau}^\ell = 1$ means uncorrupted. **AR-Diffusion** [133] advances earlier positions sooner so that they inform later predictions. **Clock Diffusion** and **Lafa** combine position-dependent progress with conditioning on a completed prefix [20, 135]. Lafa learns relative progress within each block, while Clock Diffusion also supports a sliding window in

which earlier positions finish before later ones. The order can also depend on model predictions. **TTCD** [136] chooses an ordering from the initial denoising prediction, whereas the flows underlying **Asynchronous FLM** (*a*FLM) [137] can adjust tokenwise progress throughout generation. **NeoDiff** [138] corrupts tokens with a Poisson process and estimates their progress with a learned predictor.

Committing selected predictions turns them into a part of the context. **Masked Language Flow Models** (MLFM) [139] learn to denoise undecided positions while keeping observed tokens at their clean embeddings. During generation, confident predictions are promoted to clean context and held fixed while the remaining positions continue denoising. We discuss the inference procedures in Section 4.4.

Hybrid models build discrete decisions into the generative process itself [60, 61]. **CANDI** [59] controls discrete masking separately from Gaussian corruption. During generation, unresolved positions retain continuous states, while revealed tokens become fixed context for further denoising. **Sticky Jump Diffusions** [140] incorporate this commitment through jumps from continuous states to token embeddings. Once a position jumps, its token consolidates while the other positions continue evolving. Discrete events can change the sequence length. **Expanding Generative Flows** (EFlows) [141] insert new positions initialized with Gaussian noise and continuously refine the growing sequence. Discrete decisions can also remain revisable. **HC-DLM** [142] reads out token predictions from the latent state at each step, re-noises them, and uses them to condition the next latent update. These predictions are regenerated throughout the trajectory instead of being permanently fixed.

4 Inference Strategies

A trained continuous language model leaves considerable flexibility in how its predictions are used to generate text. The sampling procedure, guidance, iterative refinement, and decoding rule can substantially affect generation quality, diversity, controllability, and computational cost. In this section, we review these inference choices and methods for efficient generation, with an overview in Figure 4.

4.1 Sampling and Time Allocation

For Gaussian interpolants, a trained model's predictions can be converted into estimates of both the velocity $\hat{b}_t$ and the score $\hat{s}_t$ (Section 3.2). These support deterministic ODE sampling and stochastic SDE sampling with drift $\hat{b}^{\epsilon}_t = \hat{b}_t + \epsilon_t \hat{s}_t$ and noise scale $\sqrt{2\epsilon_t}$. With exact velocity and score, these processes share the same time-marginal distributions [45, 143]. On a grid $0 = t_0 < \cdots < t_K = 1$, the Euler–Maruyama update is

$$\begin{aligned} \boldsymbol{x}_{t_{k+1}} &= \boldsymbol{x}_{t_k} + h_k \hat{b}^{\epsilon}_{t_k}(\boldsymbol{x}_{t_k}) \\ &\quad + \sqrt{2\epsilon_{t_k} h_k}\, \boldsymbol{\xi}_k, \qquad \boldsymbol{\xi}_k \sim \mathcal{N}(\mathbf{0}, I), \end{aligned} \tag{14}$$

where $h_k = t_{k+1} - t_k$. Setting $\epsilon_t = 0$ recovers deterministic Euler sampling, while positive values introduce stochasticity together with the corresponding score correction.

ELF [17] uses a stochastic sampler inspired by Ma et al. [144]. It evaluates the denoiser on a perturbed state at an earlier time, then uses the prediction to update the original state. A parameter controls the perturbation strength, with zero recovering Euler sampling. While this perturbation rule may not preserve the original marginal distributions, it improves sample quality in practice.

Higher-order ODE solvers can reduce integration error with fewer steps. For example, Heun's method corrects an initial Euler proposal using a second velocity evaluation. DPM-Solver++ [145] integrates the known linear part of the ODE analytically and approximates the remaining integral using denoising predictions. Its second-order multistep variant (2M) combines current and previous predictions, requiring only one new model evaluation per step. **DiffuSeq-v2** [146] applied DPM-Solver++ to text diffusion early on. In **RePlaid**'s comparison, DPM-Solver++ performs best at small budgets, while stochastic DDPM sampling becomes stronger at larger budgets [71].

For simplex-valued and spherical token representations, sampling updates must also respect the geometry of the representation. **Spherical Flows** project numerical updates back onto the sphere, while **SDM** uses reverse transitions that remain on the simplex [103, 105].

The temporal grid in Equation (14) determines where the sampler spends its evaluations. As discussed in Section 3.3, uniform spacing in a reparameterized time coordinate allocates steps nonuniformly along the original trajectory. Learned training schedules can therefore also guide inference. **CDCD** applies its learned time warp to a uniform grid, while **LangFlow** uses evenly spaced quantiles of its learned distribution over noise levels [11, 16]. **ELF** instead draws times from its training distribution and sorts them to form a randomized grid [17]. Time allocation also interacts with stochasticity. **CoBit** finds that the benefit of noise injection depends on the grid, motivating their joint selection [63].

4.2 Guidance and Control

Continuous language models can condition directly on prompts, source text, or partial generation. Guidance modifies their predictions during generation to strengthen this conditioning, improve sample quality, or favor outputs with desired properties, leveraging the continuity of states.

Originally developed for image generation, **classifier-free guidance** (CFG) [39] extrapolates from an unconditional prediction toward a conditional one. **Autoguidance** [147] uses a weaker version of the model as the reference, allowing guidance even without external conditioning. Both combine a main prediction f^{s} and a reference prediction f^{w} as

$$f^{\mathrm{g}}(\boldsymbol{x}, t) = f^{\mathrm{w}}(\boldsymbol{x}, t) + \omega\big[f^{\mathrm{s}}(\boldsymbol{x}, t) - f^{\mathrm{w}}(\boldsymbol{x}, t)\big], \tag{15}$$

where $\omega = 1$ recovers the main prediction and $\omega > 1$ extrapolates beyond it. For Gaussian interpolants, the same combination can be applied to velocities, scores, or denoised clean representations. The reference prediction can be formed by withholding information or weakening the model. **SED** removes both text context and self-conditioning, while **Lafa** removes the conditioning prefix in block autoregression [20, 51]. **FLM** obtains weaker predictions by enabling dropout

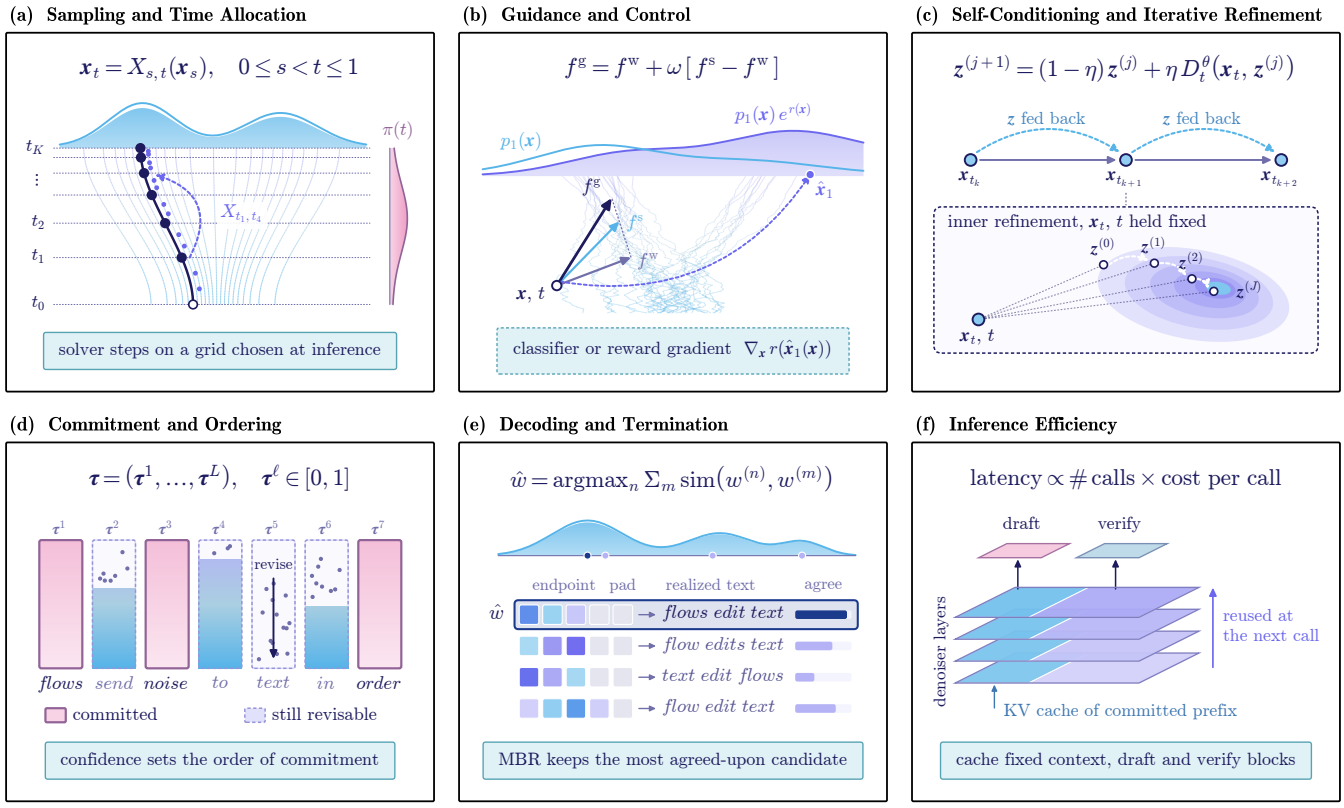


Fig. 4: Inference strategies for continuous language models. (a) A sampler takes solver steps on a time grid or jumps along the trajectory with a learned map such as X_{t_1,t_4}. The inference grid can differ from the training time distribution $\pi(t)$. **(b)** Guidance extrapolates from a weak prediction f^{w} past a strong prediction f^{s}. A classifier or reward gradient evaluated on the clean lookahead $\hat{\boldsymbol{x}}_1$ can also steer the state. **(c)** Self-conditioning feeds the latest prediction $\boldsymbol{z}$ into the next model call. Optional inner iterations refine $\boldsymbol{z}$ while the noisy state and time stay fixed. **(d)** Each position has its own time $\boldsymbol{\tau}^\ell$. Confident positions can be committed, and others can be renoised and revised. **(e)** Candidates are decoded into text with padding trimmed, and Minimum Bayes Risk (MBR) decoding returns the candidate $\hat{w}$ that agrees most with the others. **(f)** A KV cache avoids reprocessing the committed prefix, and speculative decoding advances several tokens with one verification pass.

during inference [15]. **SDM** uses an earlier checkpoint and applies guidance to logits before the softmax [103].

Guidance usually requires two model predictions per update. **Probe Guidance** [148] reduces this overhead by training a small prediction head on the main model's intermediate features, so the reference reuses the same backbone evaluation. **ELF** [17] learns a guided prediction during training, taking the guidance strength as an input [149, 150]. Here, guidance contrasts predictions with and without self-conditioning, and after training, a single network evaluation produces the guided result.

Beyond conditioning, continuous generation can be steered with gradients from a differentiable classifier or reward. **Diffusion-LM** [10] trains classifiers on noisy representations and uses their gradients to refine each denoising update. **DGLM** [97] evaluates a classifier on the predicted clean sentence embedding and differentiates through the denoiser, allowing the classifier to be trained on uncorrupted embeddings. **SSD-LM** passes soft token predictions through a pretrained text classifier's embedding layer and updates the predicted logits along the classifier gradient [40]. **TESS 2** extends this interface to reward models [151].

Furthermore, explicit constraints on text can provide guidance without a learned classifier. **Plaid** [12] constructs differentiable objectives from its predictions to favor specified tokens or spans. **Diffinity** [152] extends such control to constraints specified as regular expressions. It uses dynamic programming over a finite automaton to compute the acceptance probability of the denoising predictions. The gradient of its log probability then guides the continuous state.

It is also possible to modify model predictions directly for control. **CDCD** applies temperature and top-p truncation to its categorical predictions at each denoising step, while **Sigma** rescales the continuous score [11, 19]. These operations change the generative trajectory, whereas applying temperature or truncation only at the final token readout leaves the preceding trajectory unchanged. **BCT** [153] instead builds temperature into its spherical flows, so that both training and sampling target the tempered distribution.

4.3 Self-Conditioning and Iterative Refinement

During continuous generation, intermediate predictions provide feedback for refining subsequent estimates. Self-conditioning implements this feedback, and it can be repeated several times before advancing the noisy state. **Fixed-Point Flows** [77] interpret this as a *fixed-point iteration*

that refines the feedback prediction while holding the state and time fixed,

$$\boldsymbol{z}^{(j+1)} = (1-\eta)\boldsymbol{z}^{(j)} + \eta D_t^{\theta}\left(\boldsymbol{x}_t, \boldsymbol{z}^{(j)}\right), \tag{16}$$

where j indexes inner iterations and $0 < \eta \leq 1$. Initialization can reuse the previous sampling step's prediction, giving a warm start, or use zeros, giving a cold start. Ordinary self-conditioning corresponds to one warm-started iteration with $\eta = 1$, while $\eta < 1$ damps each update. Additional inner iterations can improve sample quality, at the cost of additional model evaluations.

Feedback also interacts with the numerical solver. In self-conditioned flows, **Untied Self-Conditioning** [154] observes that the previous prediction has already influenced the current state through the solver before it is supplied again as explicit feedback. It dampens redundant feedback directions and corrects the solver update using prediction history. These corrections are calibrated offline and do not require retraining or additional model evaluations.

Self-conditioning can also reinforce repetitive predictions. **ACE** [121] estimates a repetition direction by contrasting feedback from high- and low-repetition calibration trajectories. It then subtracts a fixed scaled offset along this direction from the self-conditioning input, reducing repetition in unconditional generation.

4.4 Commitment and Ordering

During generation, intermediate predictions can signal which positions should advance or consolidate. **Inverse-Confidence Sampling** (INCO) [155] assigns larger denoising steps to less confident positions, and finds that this improves LangFlow's sample quality without retraining. **TTCD** [136] uses tokenwise time and fixes an ordering from the entropy of its initial predictions. The asynchronous flows underlying ***a*FLM** [137] instead update priorities throughout generation using tokenwise time schedules adapted from unmasking strategies of masked diffusion models.

Instead of adjusting how fast positions advance, other methods commit selected predictions as fixed context during generation. **MLFM** [139] commits positions whose maximum token probability exceeds a threshold, replacing their states with clean token embeddings. Other positions continue along the existing trajectory. **GRAFT** [156] instead commits rare candidates early and regenerates the rest, recovering rare words that averaging over candidates suppresses. **Clock Diffusion**'s confidence-based sampler commits the longest contiguous prefix of the active window whose positions either pass a confidence threshold or complete their local denoising schedules [135]. This supports caching of completed context, which we discuss in Section 4.6.

Lastly, commitment can also be combined with revision. **x-Prediction Flow** [157] uses a learned policy to choose progress increments from predictive confidence and current progress, and commits the most confident position at each step. If a position's confidence falls substantially below its accumulated progress, the sampler moves its state back toward the mask embedding and reduces its progress, allowing the prediction to be reconsidered.

4.5 Decoding and Termination

At the end of generation, continuous predictions are decoded into text, and the decoding procedure is partly determined by the choice of representation. Some models already predict token probabilities and can return their argmax, while others require an additional decoding stage. **ELF** uses one final evaluation of the denoising network to predict all tokens in parallel, whereas **LD4LG** generates tokens autoregressively from its latents [17, 52]. Token predictions can stabilize before the continuous trajectory ends, allowing sampling to stop early. Vaina et al. [158] study stopping criteria based on low predictive entropy, small KL divergence between successive predictions, or few token changes over several consecutive steps. **ELF-REG** [78] instead executes a fixed number of initial updates on a finer time grid, then decodes its latest prediction of the clean representations. This avoids completing the trajectory without requiring few-step training.

For variable-length generation, output length requires a separate decision when the sampler evolves a fixed number of positions. **TESS** generates on a fixed-length canvas and learns to fill unused positions with padding tokens [54]. **Differmer** and **DINOISER** instead predict target lengths and generate candidates across several lengths and noise initializations [70, 159]. *Minimum Bayes Risk (MBR)* decoding [160, 161] selects among these candidates based on their agreement with one another. Blockwise generation can determine length progressively by checking each block for an end-of-sequence token before generating the next. **RangeSWA** [162] lets a trained model generate beyond its training length by combining sliding-window attention with summaries of distant blocks.

4.6 Inference Efficiency

Practical speedups depend on both the number of sequential network evaluations and the work performed in each. Caching reduces repeated processing of context, while speculative decoding uses parallel proposals to advance several tokens with one verification pass.

Caching reuses representations that remain fixed during generation. **SeqDiffuSeq** encodes its source sequence once, while blockwise models reuse the key-value (KV) cache of completed prefixes [20, 68, 86]. Extending this cache requires computing representations of newly committed tokens as conditioning inputs. **Clock Diffusion** [135] combines this computation with the next denoising pass, processing the newly committed tokens alongside the active noisy window. Its attention mask prevents clean tokens from attending to noisy positions, allowing their cache entries to be constructed without a separate forward pass.

In *speculative decoding*, a drafter proposes a token block, which an AR model verifies in one forward pass, accepting a prefix and correcting rejected predictions [163, 164]. **FlashMap** [165] uses a flow-based drafter conditioned on the target model's hidden states to propose tokens jointly. **Lafa** [20] instead drafts with one denoiser step and verifies with the same network's causal predictions, matching its own greedy AR output. Its fused implementation verifies the previous draft while preparing possible next drafts in the same pass, retaining the one corresponding to the

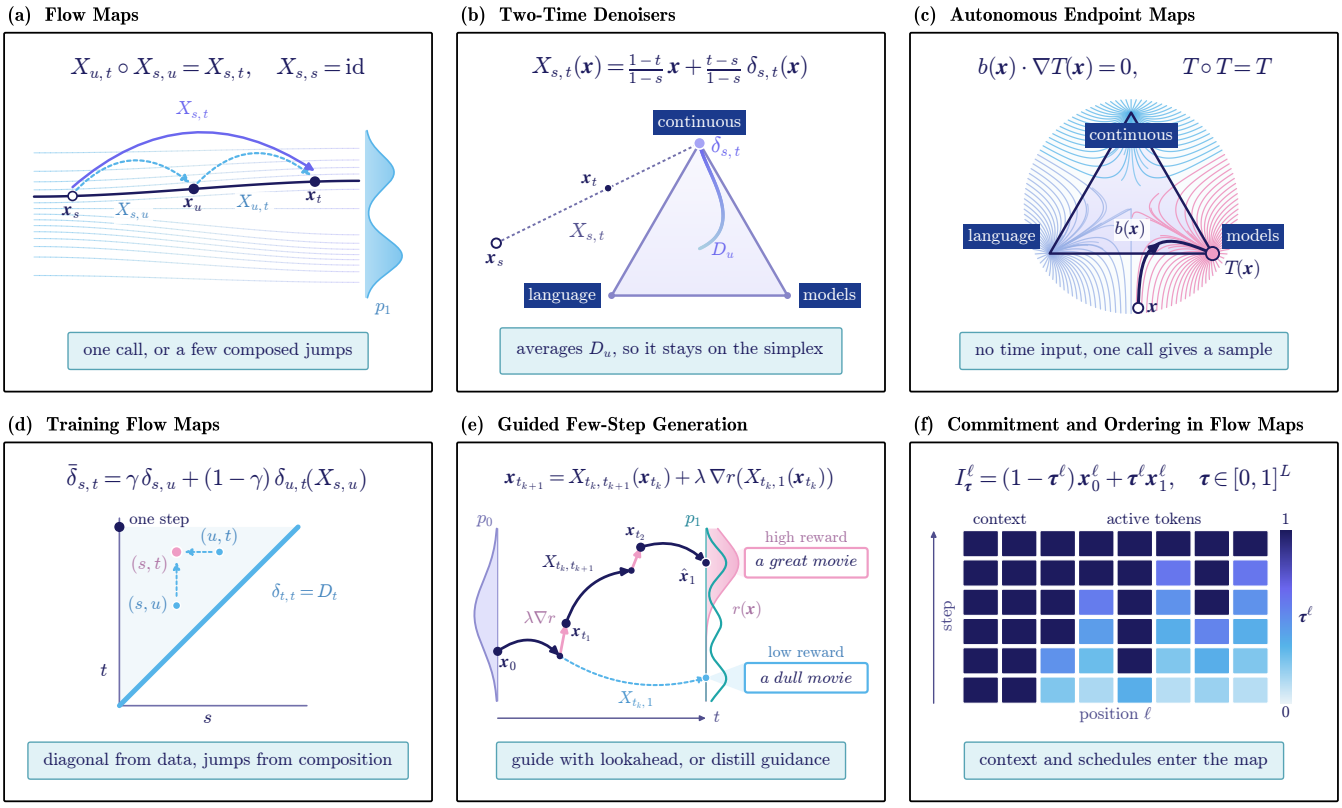


Fig. 5: Few-step generation with continuous language models. (a) A flow map jumps between any two times along the probability flow, and composing short jumps gives the long one. **(b)** The two-time denoiser $\delta_{s,t}$ is the clean estimate that a straight step must use to land on the trajectory. It averages the denoiser D_u along the path, so it stays on the simplex. **(c)** Beckmann transport uses an autonomous field b that carries every point to a vertex. Its endpoint map T needs no time input, and one call gives a sample. Pink lines mark the points that reach the vertex $T(\boldsymbol{x})$. **(d)** In self-distillation, the diagonal $s = t$ is learned from data as the denoiser D_t. Each longer jump (s,t) can be supervised by composing the shorter jumps (s,u) and (u,t) (semigroup) or other consistency objectives. **(e)** Reward guidance alternates flow map jumps with short steps along $\lambda\nabla r$, where r is evaluated on the clean lookahead $X_{t_k,1}$. In this sentiment example, the unguided path ends at a phrase with lower reward. A guided teacher can also be distilled into a few-step map. **(f)** Each position has its own time $\boldsymbol{\tau}^{\ell}$, shown by color at each generation step. Context and committed positions sit at $\boldsymbol{\tau}^{\ell} = 1$, while other positions follow their own schedules and are committed at different times.

accepted prefix. Because these methods change both the cost of each evaluation and the number of tokens it advances, latency and throughput measurements are needed to establish their practical speedups.

5 Few-Step Generative Modeling

The inference strategies of Section 4 make the most of a trained denoiser, but conventional iterative samplers still require repeated model evaluations. Their cost depends on the number of steps, the active sequence length, and whether computation can be reduced through caching or blockwise generation. This section turns to models that are *trained* to generate in one or a few steps, with an overview in Figure 5. Few-step generation is also where the fundamental advantage of continuous models discussed in Section 2.2 becomes most visible. Standard parallel discrete-diffusion samplers that sample tokens conditionally independently use a product of predicted marginals, whereas a continuous model can learn a deterministic map from noise to sequences that captures dependencies between the tokens.

5.1 Formulations and Generative Interfaces

A few-step sampler must take large steps along the path from noisy to clean states, and a single Euler step is most reliable when the model's estimate of the clean sequence changes minimally over the interval. This holds when the output is nearly determined by the state and the conditioning input. When many sequences remain plausible, the estimate is an average over them and changes as the trajectory commits to one, so a large step lands on an incoherent average rather than on any single sequence. For language, this commitment happens within a short window of the trajectory (Section 3.3), which makes naive few-step sampling inaccurate. Few-step methods therefore either reduce the error of the sampler or learn the solution of the generative ODE directly, as a map from an intermediate state to a later point on its trajectory or to its endpoint.

5.1.1 *Reducing Solver Error*

The simplest way to sample a continuous language model in a few steps is to take large steps with the denoiser it

already has. With the linear interpolant, an Euler step of the probability flow ODE (2) from time s to time t is given as

$$\boldsymbol{x}_t \approx \boldsymbol{x}_s + (t-s)b_s(\boldsymbol{x}_s) = \tfrac{1-t}{1-s}\boldsymbol{x}_s + \tfrac{t-s}{1-s}D_s^\theta(\boldsymbol{x}_s), \quad (17)$$

which moves the state along a straight line toward the current clean estimate $D_s^\theta(\boldsymbol{x}_s)$. Early few-step text generation relied on making the iterative flow sampler robust to large steps, mostly in sequence-to-sequence tasks where the conditioning largely determines the output. **Differmer** [70] showed that an embedding diffusion model that predicts clean embeddings, combined with a cross-entropy "anchor" loss and a rescaled noise schedule, retains its quality when generation is reduced to a few dozen steps.

Flow matching then popularized the *linear interpolant* between noise and data, which tends to produce straighter sampling trajectories than the variance-preserving paths of diffusion models, improving its sample quality when taking fewer and larger Euler steps [43–45]. **FlowSeq** [166] trains a velocity field with the same anchor loss and generates with a single Euler step, **FMSeq** [167] combines embedding denoising, self-conditioning, and a shifted time input to sample in just ten steps, and **LF** [168] integrates a flow in the latent space of a text autoencoder with ten Euler steps. **FlowLM** [169] later adapted a pretrained embedding diffusion model to the linear interpolant, so that three to five Euler steps suffice. These methods reduce the number of steps substantially when the output is nearly determined by the input, as in paraphrasing or question generation, but their one-step outputs degrade on open-ended tasks such as dialogue, where many responses are plausible. Under an independent noise-data coupling, the Bayes-optimal denoiser at pure noise predicts the conditional mean of all possible outputs. A single Euler step using an accurately learned denoiser therefore tends to blend distinct responses when many completions are plausible, motivating the explicit learning of few-step generators.

5.1.2 Flow Maps

The **flow map** framework [21, 170] replaces the instantaneous velocity with the *two-time solution operator* $X_{s,t}$ of the probability flow ODE (2), which transports a state at time s to the state at time t along the flow trajectory, $X_{s,t}(\boldsymbol{x}_s) = \boldsymbol{x}_t$. It is characterized by any of the three equivalent *consistency conditions*,

$$\textit{Lagrangian:} \quad \partial_t X_{s,t}(\boldsymbol{x}) = b_t\big(X_{s,t}(\boldsymbol{x})\big), \quad (18)$$

$$\textit{Eulerian:} \quad \partial_s X_{s,t}(\boldsymbol{x}) + b_s(\boldsymbol{x}) \cdot \nabla X_{s,t}(\boldsymbol{x}) = 0, \quad (19)$$

$$\textit{Semigroup:} \quad X_{u,t} \circ X_{s,u} = X_{s,t}, \quad (20)$$

with the identity condition $X_{s,s} = \mathrm{id}$. The semigroup condition determines the map only together with the *tangent condition* $\partial_t X_{s,t}|_{t=s} = b_s$, which states that the map at a single time s is equivalent to the instantaneous velocity. A single evaluation of $X_{0,1}$ transports noise to a clean sample $X_{0,1}(\boldsymbol{x}_0) = \boldsymbol{x}_1$, and chaining $X_{t_k,t_{k+1}}$ over a discrete time grid $\{t_k\}_{k=0}^K$ trades inference steps for quality. Earlier few-step frameworks are special cases of flow maps. Consistency models [171] learn only the jumps from noise to the data $X_{s,1}$, progressive distillation [172], consistency trajectory models [173], and shortcut models [174] learn jumps between arbitrary times or of a given step size, and MeanFlow [149] and TVM [175] learn the average velocity $v_{s,t}$ with $X_{s,t}(\boldsymbol{x}) = \boldsymbol{x} + (t-s)v_{s,t}(\boldsymbol{x})$. Since $X_{s,t}$ is the exact solution operator of the ODE, a jump of any size is free of discretization error of (17), making it a suitable modeling target for few-step generation.

5.1.3 Two-Time Denoisers

For language, a main design question is how to learn the solution operator of the continuous flow in the ambient space while respecting the geometry of the targets on the product of simplices $(\Delta^{V-1})^L$. **Categorical Flow Maps** (CFM) [22] and **Flow Map Language Models** (FMLM) [15] concurrently introduced, and **Discrete Flow Maps** (DFM) [23] further developed, a reparameterization of the flow map through a *two-time denoiser* $\delta_{s,t}$, also called the *mean denoiser*,

$$X_{s,t}(\boldsymbol{x}) = \tfrac{1-t}{1-s}\,\boldsymbol{x} + \tfrac{t-s}{1-s}\,\delta_{s,t}(\boldsymbol{x}), \quad \delta_{s,s}(\boldsymbol{x}) = D_s(\boldsymbol{x}). \quad (21)$$

Comparing with (17), the two-time denoiser is the clean estimate that a single straight step must use in place of D_s to land exactly on the trajectory at time t. Solving the ODE with $b_u(\boldsymbol{x}) = (D_u(\boldsymbol{x}) - \boldsymbol{x})/(1-u)$ shows that it is a weighted average of the denoiser along the trajectory,

$$\delta_{s,t}(\boldsymbol{x}) = \int_s^t w_{s,t}(u) D_u\big(X_{s,u}(\boldsymbol{x})\big)\,du, \quad (22)$$

where $w_{s,t}(u) = \frac{(1-s)(1-t)}{(t-s)(1-u)^2}$ is positive and integrates to one over $[s,t]$ for $t < 1$, and $\delta_{s,1}$ is the limit as $t \to 1$. For one-hot data, every D_u lies in the product of simplices $(\Delta^{V-1})^L$. Since $\delta_{s,t}$ is a convex combination of these denoisers, it lies in $(\Delta^{V-1})^L$ as well. A per-position softmax is thus a natural parameterization, as $\delta_{s,t}$ always lies on the simplex, and the losses for its training can be written as cross-entropies and KL divergences between categorical distributions (Section 5.2). The same parameterization applies in learned embedding and latent spaces, where $\delta_{s,t}$ predicts a clean embedding and is trained by regression [77, 176].

5.1.4 Pushforward Maps

Flow maps take two-time inputs (s,t) so that one network can be used to solve the ODE in any number of steps. A simpler formulation drops time altogether and learns a single *pushforward generator* G satisfying $G_{\#}p_0 = p_1$ that maps noise directly to a clean sequence, $\hat{\boldsymbol{x}}_1 = G(\boldsymbol{x}_0)$ with $\boldsymbol{x}_0 \sim p_0$. Examples include **DLM-One** [177], **Coupling Models** [178], and **drifting**-type text generators [179, 180]. The same noise-to-sequence map also appears inside continuous AR models such as **CALM** [93], **PTP** [181], and **Gumbel Distillation** [182], where a prediction head maps continuous noise to the next block of tokens. Since these types of generators are not tied to a trajectory, they are trained with distributional objectives or on explicit pairs of noisy and clean sequences (Section 5.2), and any additional function evaluations must come from a separate refinement procedure (Section 5.3).

5.1.5 Autonomous Endpoint Maps

Another approach to remove time conditioning is to remove it from the flow dynamics. **Beckmann Transport Models** (BTM) [183] replace the time-dependent velocity by

the *autonomous* field obtained by averaging the conditional velocity over time,

$$b(\boldsymbol{x}) = \mathbb{E}_{t,\boldsymbol{x}_0,\boldsymbol{x}_1}\big[\dot{I}_t \mid I_t = \boldsymbol{x}\big], \quad t \sim \mathcal{U}[0,1] \tag{23}$$

which satisfies the *divergence constraint* $\nabla \cdot (\nu\, b) = p_0 - p_1$ where $\nu = \int_0^1 p_t dt$. If the data lies on a lower-dimensional set M_1, such as the vertices of a simplex, integrating $\dot{X} = b(X)$ from any starting point reaches M_1 at some stopping time $\tau(\boldsymbol{x}_0) \in [0, \infty)$ under proper conditions. The long-time limit of the autonomous flow gives the *endpoint map* $T(\boldsymbol{x}) = \lim_{r\to\infty} X_r(\boldsymbol{x})$, where X_r is the flow of b after integration time r. This map satisfies the first-order *conservation equation*

$$\begin{cases} b(\boldsymbol{x}) \cdot \nabla T(\boldsymbol{x}) = 0 & \boldsymbol{x} \notin M_1 \\ T(\boldsymbol{x}) = \boldsymbol{x} & \boldsymbol{x} \in M_1 \end{cases}. \tag{24}$$

The map T is constant along the flow trajectories of b, idempotent such that reapplying a converged map produces the same fixed point $T \circ T = T$, and needs no time input. **Discrete Beckmann Transport Models** (DBTM) [24] bring this framework to language with ambient flows toward one-hot vectors and a per-position softmax parameterization. Unlike a flow map, a single call of the endpoint map is the natural sampler for clean sequences, and additional calls are spent on refinements that renoise and remap part of the sequence instead of jumping along a flow ODE (Section 5.5).

5.2 Learning Objectives and Training Strategies

5.2.1 Training Flow Maps

Each of the conditions (18)-(20) for the flow map yields a training objective. In **self-distillation** [21], the diagonal time pairs $s = t$ in the (s,t) time grid are trained by flow matching, which with the two-time denoiser is equivalent to training the ordinary instantaneous denoiser $\delta_{t,t} = D_t$, and then the diagonal in turn supervises the off-diagonal pairs $s \neq t$ through one of the three conditions (18)-(20), with the target placed under a stop-gradient. With the two-time denoiser, the semigroup condition becomes a statement about convex combinations. For any intermediate time $s \le u \le t$,

$$\delta_{s,t}(\boldsymbol{x}) = \gamma\, \delta_{s,u}(\boldsymbol{x}) + (1-\gamma)\delta_{u,t}\big(X_{s,u}(\boldsymbol{x})\big), \tag{25}$$

with $\gamma = \frac{(1-t)(u-s)}{(1-u)(t-s)} \in [0,1]$, so composing two short jumps defines a categorical target $\bar{\delta}_{s,t}$ for the long jump. The map is then trained with the following cross-entropy objective,

$$\begin{aligned} \mathcal{L}(\delta) = \mathbb{E}_{s,u,t}\Big[\textstyle\sum_\ell \mathrm{CE}\big(\mathrm{sg}(\bar{\delta}^\ell_{s,t}),\, \delta^\ell_{s,t}(I_s)\big)\Big] \\ + \mathbb{E}_t\Big[\textstyle\sum_\ell \mathrm{CE}\big(\boldsymbol{x}_1^\ell,\, \delta^\ell_{t,t}(I_t)\big)\Big], \end{aligned} \tag{26}$$

where the diagonal target $\boldsymbol{x}_1$ can be replaced by the denoiser of a pretrained teacher. **FMLM** [15] introduced this semigroup objective. **CFM** [22] instead trains the off-diagonal with *endpoint-consistent Lagrangian distillation*, a cross-entropy consistency term on the endpoint prediction combined with a penalty on its time derivative, which together bound the Lagrangian residual. **DFM** [23] rewrites all three conditions (18)-(20) as fixed-point equations for $\delta_{s,t}$, constructing the Lagrangian and Eulerian targets in logit space since they can leave the simplex.

The objectives differ in cost and stability. The semigroup objective needs only forward evaluations of the network, whereas Eulerian needs a *Jacobian-vector product* (JVP) through the full $L \times V$ state, and Lagrangian needs a time derivative of the output. While existing work finds the semigroup form preferable for one-step quality [15, 18, 23], a universally optimal objective is yet to be settled. More generally, training is sensitive to balancing the diagonal and off-diagonal terms in the loss, and common practices include splitting each batch between the two, adaptive loss weighting [149, 184], sampling times (s,t) following time reparameterizations described in Section 3.3, and sampling the full jump $(s,t) = (0,1)$ at a fixed frequency so that the one-step map is trained directly.

5.2.2 Distillation and Self-Distillation

In flow map distillation, the diagonal can be supervised with data, a pretrained teacher, or both. **FMLM** [15] and **Lafa** [20] distill a pretrained flow model, using the teacher's denoiser on the diagonal and self-distillation off the diagonal. **FMLM*** [77] and **CoFlow** [176] distill self-conditioned embedded flows (Section 5.5), where CoFlow shows that the embedding itself can be adapted during distillation, and **Gumbel Straight Flow** (GSF) [185] distills an AR model. **CFM** [22], **DFM** [23], the scaled CFM of Davis et al. [18], and **FMLM+** [186] instead learn the denoiser from data and distill it within the same model, either jointly or in a second stage, so that no separate teacher is needed. Outside the two-time denoiser parameterization, **ELF** [17] uses progressive distillation [172] to repeatedly cut the number of steps of an embedded flow in half down to one step, **Sigma** [19] uses parallel decoding distillation [187] to compress large blockwise embedding diffusion into a few steps, and Warczyński et al. [188] use shortcut modeling [174] to learn embedded velocity from data and self-distill it into the average velocity. In general, a teacher model provides converged, low-variance targets at the cost of a pretraining stage, whereas self-distillation is cheaper but must learn the diagonal and the jumps that depend on it simultaneously by matching a moving target.

5.2.3 Training Pushforward Maps

Instead of enforcing consistency along trajectories, *distributional objectives* compare the distribution of the generated outputs with that of the data or of a teacher,

$$\min_\theta \mathcal{D}\big(G_{\theta\#}p_0, p_1\big), \tag{27}$$

where $\mathcal{D}$ is a divergence or a scoring rule that can be estimated from samples. The methods below differ in the choice of $\mathcal{D}$ and in how its gradient is estimated. **Score-based distillation** [189, 190] moves each generated sample along the difference between a frozen teacher denoiser and a *fake* denoiser fitted online to the generator's own samples. **DLM-One** [177] applies score identity distillation with an adversarial term to embedding diffusion for sequence-to-sequence tasks. Kiem et al. [191] apply *distribution matching distillation* (DMD) to unconditional continuous language models and exploit the student's categorical outputs in two ways. **Simplex-DMD** passes pathwise gradients through continuous relaxations of the tokens, while **Reinforce-DMD**

samples discrete tokens and estimates the gradient with REINFORCE [192] and a learned density ratio. Simplex-DMD performs best with very few steps, whereas Reinforce-DMD performs best at larger budgets. **CALM** [93] trains the one-step generation head of its continuous AR model with the energy score, a strictly proper scoring rule [109] that needs only generated and data samples, with no likelihood or teacher.

Drifting Models [193] regress the generator onto a stop-gradient target that moves each sample along a drifting field $\boldsymbol{f}$,

$$\mathcal{L}(\theta) = \mathbb{E}_{\boldsymbol{x}_0 \sim p_0} \left\| G_\theta(\boldsymbol{x}_0) - \mathrm{sg}\big(G_\theta(\boldsymbol{x}_0) + \boldsymbol{f}(G_\theta(\boldsymbol{x}_0))\big) \right\|^2,$$
$$\boldsymbol{f}(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{y} \sim p_1}\big[k(\boldsymbol{x}, \boldsymbol{y})(\boldsymbol{y} - \boldsymbol{x})\big] - \mathbb{E}_{\boldsymbol{y}' \sim q_\theta}\big[k(\boldsymbol{x}, \boldsymbol{y}')(\boldsymbol{y}' - \boldsymbol{x})\big],$$

where $q_\theta = G_{\theta\#} p_0$ is the generated distribution and k is a normalized similarity kernel. The first term of $\boldsymbol{f}$ attracts each sample toward nearby data, and the second repels it from other generated samples. The field vanishes when $q_\theta = p_1$, so training drives the generated distribution toward the data distribution. **Categorical Drifting** [194] adapts this to categorical data as a mirror-descent step that adds the field to the logits and trains with cross-entropy. **R-WGF** [179] derives drifting updates as Wasserstein gradient flows on the simplex, and uses them to fine-tune a pretrained flow language model into a one-step generator. **Seq-Drifting** [180] attracts each position toward the likely continuations proposed by a frozen AR model run on the generator's own outputs, with repulsion terms that discourage collapse and repetition. Drobyshevskiy et al. [195] instead minimize the *maximum mean discrepancy* (MMD) between token-level features of generated and data sequences under a frozen flow language model, which yields a similar attraction-repulsion gradient, and use it to turn embedded flows into one-step generators.

Many distributional objectives compare full sequences through a teacher, a reference model, or a feature space, so they target the joint distribution directly and do not depend on how noise and data are paired. The tradeoff is that their quality is often bounded by the chosen reference model, and they tend to be mode-seeking, so diversity has to be monitored (Section 7).

5.2.4 Training Autonomous Endpoint Maps

Autonomous endpoint maps can be trained to satisfy the conservation equation (24) without a teacher and without access to b itself. Since b enters linearly and $b(\boldsymbol{x}) = \mathbb{E}[\dot{I}_t \mid I_t = \boldsymbol{x}]$, the per-sample velocity of the interpolant $\dot{I}_t$ can replace it within the expectation, and **DBTM** [24] minimizes the error of a stop-gradient fixed-point iteration,

$$\mathcal{L}_{\mathrm{t}}(\theta) = \mathbb{E}_{t,\boldsymbol{x}_0,\boldsymbol{x}_1} \left\| T_\theta(I_t) - \mathrm{sg}\big(T_\theta(I_t) + \dot{I}_t \cdot \nabla T_\theta(I_t)\big) \right\|^2, \quad (28)$$
$$\mathcal{L}_{\mathrm{b}}(\theta) = \mathbb{E}_{\boldsymbol{x}_1}[\mathrm{CE}(\boldsymbol{x}_1, T_\theta(\boldsymbol{x}_1))], \quad (29)$$
$$\mathcal{L}_{\mathrm{a}}(\theta) = \mathbb{E}_{t,\boldsymbol{x}_0,\boldsymbol{x}_1}[\mathbf{1}[t > t^\star]\mathrm{CE}(\boldsymbol{x}_1, T_\theta(I_t))], \quad (30)$$

where the directional derivative is computed with JVP and the stationary points satisfy $b \cdot \nabla T = 0$. Constant maps also satisfy this condition, so the transport loss (28) is combined with a boundary cross-entropy (29) that enforces $T(\boldsymbol{x}) = \boldsymbol{x}$ on clean data and an anchor cross-entropy (30) toward $\boldsymbol{x}_1$ at later times $t > t^\star$, where $t^\star$ is derived as the commitment time where the posterior over the vocabulary has committed to a single token. The resulting model is trained from scratch directly with interpolants I_t constructed from data and without requiring a teacher flow or self-distillation.

5.2.5 Noise-Data Coupling

Under an independent coupling of noise and data, many different sequences are reachable from nearby noise vectors, the probability flow is curved, and the map that compresses it is harder to learn. A good pairing can thus make few-step learning easier by straightening the flow [44, 196]. In language modeling, an effective source of couplings can be an AR model. Sampling an AR model with the Gumbel-max trick is a deterministic map from noise to text, and for a given text, noise consistent with it can be drawn with a single forward pass of an AR model. **PTP** [181] and **Gumbel Distillation** [182] distill AR into one-step models by directly regressing the noise-to-text mapping given by the AR coupling. **GSF** [185] instead distills an AR model into a flow map of the flow induced by AR coupling. The authors find that the pairs of Gaussian noise and one-hot sequences induced by AR coupling produce straighter empirical paths, and then use this coupling to supervise the diagonal of an FMLM-style flow map initialized from the weights of the AR teacher. Suitable couplings between noise and data for one-step generation can also be learned, like in **Coupling Models** [178], which fit an encoder and a normalizing flow that pair each sequence with a Gaussian latent to train a one-step decoder, or defined with the model itself, like **DBTM** [24], which match noise with targets produced by the model's own rollouts. **ReLF** [197] similarly retrains on its own rollouts over several rounds, pulling the weights back toward the base model after each round to limit drift.

5.3 Sampling with Learned Maps

A learned flow map generates a sequence in one evaluation of $X_{0,1}$ or through several shorter transitions. Given a time grid $0 = t_0 < \cdots < t_K = 1$, deterministic sampling applies

$$\boldsymbol{x}_{t_{k+1}} = X_{t_k, t_{k+1}}(\boldsymbol{x}_{t_k}). \quad (31)$$

For an exact flow map, composition gives the same result as a single transition. With a learned map, the number and placement of transitions affect approximation error and generation quality.

Sampling can also combine learned transitions with fresh noise. **FMLM** [15] adopts γ-sampling from **Consistency Trajectory Models** [173]. Each update jumps to a time at or beyond the next grid point, then adds noise to return to the scheduled corruption level. The parameter $\gamma \in [0, 1]$ controls the amount of fresh noise. Setting $\gamma = 0$ gives deterministic map composition, while $\gamma = 1$ predicts a clean sequence at every step and noises it again before the next prediction, as in multi-step consistency sampling [171].

Endpoint maps return a clean proposal at every model evaluation, so additional computation takes the form of a separate refinement procedure. **DBTM** [24], for example, fixes selected tokens as context and replaces unresolved positions with fresh noise before applying the map again. We discuss these feedback procedures in Section 5.5. Distribution-matching students also return a clean sample

at every evaluation, but they condition on the noise level, so additional steps can follow a schedule of decreasing noise levels. Kiem et al. [191] train such students for continuous language models. Among the transitions they test, *forward renoising*, which adds fresh Gaussian noise to each clean prediction, gives the best tradeoff between generative perplexity and entropy.

5.4 Guided Few-Step Generation

Guidance is one of the main motivations for continuous language models (Section 2.3), but its standard forms such as **classifier-free guidance** [39] and **autoguidance** [147] act on the instantaneous dynamics (Section 4.2). A flow map never evaluates the velocities it jumps over, so these ideas do not apply directly. Flow maps are instead trained to reproduce guided dynamics or steered through their own predictions at generation time.

5.4.1 Distilling Guided Flow into Flow Map

The most direct approach distills a teacher whose dynamics already include guidance [198, 199], so that the student reproduces the solution curve of the guided flow. **ELF** [17] distills the self-conditioned guided flow (Section 4.2) into a few-step flow map student conditioned on ω through *progressive distillation*. **FMLM*** [77] trains a single flow map conditioned on the guidance weight of its self-conditioned teacher, sampling one weight per example and sharing it between teacher and student, and **CoFlow** [176] incorporates label guidance into the target from which its map is distilled. **Sigma** [19] distills a teacher that combines guidance with a score temperature, which together define the score of a tempered and guided distribution. While distilling at a fixed weight is simpler, conditioning on the guidance weight makes it possible to control the tradeoff between quality and diversity without retraining, which is useful for evaluation (Section 7).

For simplex-valued representations, guidance introduces an additional complication: extrapolating between two categorical denoisers can produce predictions outside the probability simplex. Such guided predictions cannot directly serve as targets for cross-entropy training. One option is to retain separate predictions for the strong and weak denoisers, using two heads or model evaluations, and combine them for guidance. Alternatively, the guided field can be distilled directly using an unconstrained regression objective, sacrificing the convenient cross-entropy formulation.

5.4.2 Guiding the Flow Map

A trained flow map can also be steered directly. Since $X_{t,1}$ predicts the clean sequence from any intermediate state, a reward r that acts on clean sequences can be evaluated on a nearly clean lookahead from an intermediate state $\boldsymbol{x}_t$, and its gradient can be propagated back through the map. **Flow Map Reward Guidance** (FMRG) [200], applied to language by **FMLM** [15], alternates a step of the flow map with a step along the gradient of the reward evaluated on the lookahead,

$$\boldsymbol{x}_{t_{k+1}} = X_{t_k,t_{k+1}}(\boldsymbol{x}_{t_k}) + \lambda \nabla_{\boldsymbol{x}_{t_k}} r\big(X_{t_k,1}(\boldsymbol{x}_{t_k})\big), \tag{32}$$

where λ controls the strength of guidance. This enables reward guidance with only a few steps and without access to the velocity field. **CFM** [22] applies the same idea to categorical image generation. For time-independent generators, steering acts on the noise or the logits. **Coupling Models** [178] use gradient ascent on the latent with respect to a classifier, and apply CFG on the logits of a conditional model. All of these methods rely on the differentiable state discussed in Section 2.3. Discrete-state models must reweight candidates or approximate the effect of a jump on the reward, whereas a flow map provides both a clean prediction and a differentiable path to it in a single evaluation.

5.5 Few-Step Generation with Iterative Feedback

Iterative samplers for language rarely integrate a plain ODE. They reuse predictions through self-conditioning, commit or revise tokens according to their confidence, and increasingly let different positions advance at different rates (Section 4). A few-step model needs to absorb these mechanisms into its learning, either by integrating them into the definition of the map or by enlarging the state on which the map acts.

5.5.1 Self-Conditioning in Flow Maps

Self-conditioning feeds the denoiser its own previous prediction $\boldsymbol{z}$ of the clean sequence, $D_t(\boldsymbol{x}, \boldsymbol{z})$, and is one of the most effective components of continuous language models (Section 3.4). However, ordinary self-conditioning introduces history dependence, so the dynamics are no longer closed in the continuous state $\boldsymbol{x}$ alone. A flow-map formulation requires an augmented state or an appropriate closure, such as fixed-point refinement. **FMLM*** [77] resolves this by observing that the minimizer of the self-conditioned objective is the ideal denoiser regardless of $\boldsymbol{z}$. Iterating $\boldsymbol{z} \leftarrow D_t(\boldsymbol{x}, \boldsymbol{z})$ is thus a fixed-point iteration that refines the estimate. Replacing $\boldsymbol{z}$ with its fixed point yields a fixed-point flow whose velocity depends on $\boldsymbol{x}$ and t alone and thus has a well-defined flow map. FMLM* compresses both dimensions, distilling the fixed-point iteration into a single denoiser that takes no self-conditioning and then distilling its flow into a flow map. **CoFlow** [176] applies the same idea in a jointly trained latent space, and other works either keep self-conditioning in a distilled model [17] or disable it [19, 20]. The common takeaway is that feedback must either become part of the map's definition or be removed from the dynamics before the map is learned.

5.5.2 Commitment and Ordering in Flow Maps

A perfectly learned flow map samples the data distribution exactly, but in practice, the procedure in which tokens are determined affects both learnability and quality. Under the standard linear interpolant (Section 2.4), all positions are resolved within a short time window, so its flow map must learn to commit the whole text at once. Resolving some tokens first turns the rest into easier conditional problems, as in AR and masked models [131, 132, 201]. In flows, this can be expressed by giving each position its own time [133, 134] (Section 3.6). With this, **FMLM+** [186] and **DBTM** [24] train with a *partial-context interpolant* with a random subset of positions clean at $t = 1$ and the rest at a shared intermediate time. This makes any-order sampling possible

with clean context conditioning, and can be initialized from a pretrained masked model [186]. Conditional and block-autoregressive flow maps [19, 20, 185] can be interpreted as special cases where a block of tokens is denoised to an intermediate time t while preceding blocks are fixed.

These models can generate in rounds, retaining confident predictions as context for the next proposal. **FMLM+** produces a complete candidate in one or a few flow-map steps and reads token confidence from the final map output. It commits tokens whose maximum probability exceeds a threshold and restarts the remaining positions from fresh noise. This confidence accompanies a jointly generated candidate, whereas confidence-based masked decoding [202] uses token marginals predicted from partial context. **DBTM** uses a similar refinement loop with its time-independent endpoint map. It selects tokens using either their maximum predicted probability or a small head trained to predict token accuracy from the map's hidden states. The latter requires no additional model evaluation. Both methods enforce a minimum number of new commitments per round to prevent stalling.

Ordering can also be incorporated into the learned map. **TTCD** [136] learns shortcut models conditioned on starting and ending tokenwise times. It supervises a long transition with the composition of two shorter ones while retaining the denoising objective for infinitesimal steps. At generation time, it fixes token priorities from the entropy of the initial prediction, allowing the shortcut to compress an ordered trajectory. **Asynchronous FMLM** (aFMLM) [137] learns maps of asynchronous flows with prescribed or prediction-dependent tokenwise schedules. A one-step generator can thereby internalize the effect of an ordered trajectory without executing the scheduler. **Expanding Flow Maps** (EFM) [141] further support sequences that grow during generation, combining insertion of new noisy positions with learned transport of the representations.

6 Scaling Continuous Language Models

The previous sections focus on the theoretical and algorithmic advances of continuous language models, which have mostly been developed and validated at small scale. Since much of the success of AR language models comes from scaling, a central question for continuous models is whether these advances carry over to larger models. We first review early attempts, which led continuous models to be regarded as less efficient than discrete ones, and then recent work that revisits this view and scales them to billions of parameters. Table 2 lists the parameter count, training tokens, and initialization of each model.

Model	Params	Tokens	Initialization
Plaid [12]	1.3B	314B	Scratch
LCM [98]	7B	2.7T	Scratch
RePlaid [71]	1.7B	≈524B	Scratch
Scaling CFM [18]	1.7B	2.1T	Scratch
Cola DLM [86]	1.8B	≈185B	Scratch
PLM [206]	1.3B, 3B	300B	Scratch
SSD-2 [128]	13B	38B	OPT [205]
TESS 2 [151]	7B	45B	Mistral-7B [207]
Sigma [19]	3B, 8B	300B	NLD [208] AR stage
Lafa [20]	0.5B, 7B, 8B	<30B	Qwen2.5/3 [209, 210]
DSL-LLaDA [211]	8B	≈66M	LLaDA [14]
LLaCO [212]	8B	≈1.7B	NLD [208]

TABLE 2: Scale of continuous language models. Tokens denote the number of pretraining or continued-training tokens. ≈ marks estimates from the reported training setup or compute.

6.1 Early Attempts at Scaling

Plaid [12] gave the first compute-optimal scaling analysis for diffusion language models, comparing them with AR models trained under the same compute budgets. It introduced algorithmic improvements over earlier diffusion models, including a learned noise schedule (Section 3.3) and a categorical parameterization of the denoiser, and showed that a continuous language model can outperform a small AR model such as GPT-2 in likelihood-based evaluation. However, its scaling analysis showed that continuous language models still need about $64\times$ more training compute than AR models to reach the same likelihood, and this gap did not shrink as models were scaled up.

Later work on discrete diffusion measured its own scaling using this gap as a reference point. In particular, Nie et al. [13] found that masked diffusion scales at the same rate as AR models, and that its compute gap to AR models is several times smaller than that of Plaid. Discrete diffusion was then rapidly scaled to billions of parameters [14, 203] and to production-scale models [204]. Together, these results shifted the field toward discrete-state diffusion, while continuous-state models were considered harder to scale.

Nevertheless, a few works scaled continuous models to billions of parameters. **SSD-2** [128] scales SSD-LM [40] up to 13B parameters, initializing from pretrained OPT [205] models. It also proposes inference-time model composition for continuous language models, which averages the predicted logits of a billion-parameter model and a small model customized by the user at every denoising step. Since each denoising step revises the active block, the small model can influence every position, and the ensemble outperforms an ensemble of AR models, which can only combine their predictions one token at a time. Similarly, **TESS 2** [151] scales TESS [54] to 7B parameters by adapting a pretrained Mistral-7B. During this adaptation, the authors switch the model from causal to bidirectional attention but keep the shifted next-token targets of the AR model, and then apply instruction tuning. **LCM** [98] trains a 7B model from scratch, generating sentence embeddings by diffusion and decoding them with a frozen AR decoder (Section 3.1.4).

6.2 Recent Progress

Recent advances such as few-step generation (Section 5) and latent models (Section 3.1.4) have renewed interest in scaling continuous-state language models.

6.2.1 *Training from Scratch*

RePlaid [71] revisits the comparison of Plaid [12] under matched conditions and a modernized architecture. Specifically, retaining Plaid's basic probabilistic formulation, it adopts the DiT backbone of recent discrete diffusion models (Section 2.1) and compares it against discrete diffusion and AR models. Under this setup, the reported gap shrinks

from approximately $64\times$ to $20\times$ with self-conditioning ($27\times$ without), over the evaluated compute range and continuous diffusion scales at the same rate as discrete diffusion, which suggests that much of the gap reported by Plaid stemmed from the outdated architecture and training setup rather than from the continuous formulation itself. Notably, when models are trained past their compute-optimal budget to reduce inference costs [213], RePlaid consistently achieves lower loss than masked diffusion.

Other studies have scaled continuous formulations that were originally evaluated at small scales to billions of parameters. Davis et al. [18] scale one-hot-based CFM [22] (Section 5.1) to 1.7B parameters through design-space studies and hyperparameter searches over the time schedule, loss weighting, and optimizer settings. To handle the vocabulary-sized one-hot inputs and logits at scale, they adopt a memory-efficient cross-entropy implementation, and also show that keeping a random prefix clean is essential for prompted-completion quality in the reported evaluation. The few-step advantage of flow maps also carries over to this scale, as the self-distilled model improves the tradeoff between quality and speed over discrete diffusion baselines. Guo et al. [86] similarly study how the VAE latent space of **Cola DLM** behaves at scale. A fixed latent space is competitive at small budgets but saturates early as compute grows, whereas latents trained jointly with the denoiser from a stable VAE initialization keep improving.

In addition, **Parallel Language Models** (PLMs) [206] train the pushforward generator of PTP [181], which maps noise to a block of tokens in one forward pass (Section 5.1), from scratch at up to 3B parameters. At 1.3B parameters, PLMs are compared against a separately trained, matched AR baseline. At 3B parameters, the comparison instead uses the model's own autoregressive decoding mode.

6.2.2 Adapting Pretrained Models

While most scaling works focus on fixed-length unconditional generation and likelihood-based benchmarks, recent works start from large pretrained models and evaluate them extensively on downstream tasks. **Sigma** [19] scales the embedding diffusion of **Plaid** and **RePlaid** to 3B and 8B parameters by warm-starting from a pretrained AR model. It processes the clean prefix with causal attention while using bidirectional attention for the denoising block, together with an auxiliary next-token loss that stabilizes the training. The resulting model performs comparably to a masked diffusion model of similar scale. **Lafa** [20] scales flow language models [15, 16] to 8B parameters by adapting pretrained AR models to the same block structure, and it learns a token-wise noise schedule within each block (Section 3.6). The authors show that, with enough inference steps, flow decoding surpasses the accuracy of AR decoding, and flow map distillation improves performance with one or two steps. **Lafa** also proposes *self-speculative decoding*, where its own next-token predictions verify one-step drafts from the flow, improving the tradeoff between quality and speed.

DSL-LLaDA [211] instead starts from the masked diffusion model LLaDA [14] and replaces its mask inputs with continuous per-token Gaussian noise through continued pretraining. The adapted model evolves all positions jointly and improves few-step performance on generative tasks. **LLaCO** [212] adapts the AR–diffusion hybrid model NLD [208] to continuous states at 8B parameters, and it follows spherical geodesics as in **S-FLM** [58] (Section 3.1.6) but toward a noise endpoint that is biased to the mask embedding. It also applies time reparameterization to concentrate on the interval where token identities change (Section 3.3).

7 Applications and Evaluation

Concrete evaluation settings are essential for measuring progress in continuous language modeling and identifying its practical advantages. In this section, we review the main tasks, datasets, and evaluation metrics, many of which are shared with discrete diffusion models. We begin with unconditional language modeling, examining the limitations of sample-based metrics and alternative evaluations, then turn to conditional generation, reasoning tasks with verifiable answers, structured puzzles that test joint dependencies between tokens, and biological and scientific sequence design, as illustrated in Figure 6.

7.1 Unconditional Language Modeling

Unconditional generation removes task-specific conditioning and is therefore the most direct test of how well a continuous model fits a distribution of text. We first describe the datasets used in this setting and then the metrics used to compare sample quality and diversity.

7.1.1 Tasks and Datasets

Unconditional generation on **LM1B** [214] and **OpenWebText** [215] is a standard testbed for continuous language models in academic settings. Three key results have been reported in the literature. First, in the many-step regime, one-hot and embedding flows [15–17] now match or exceed masked and uniform discrete diffusion at comparable scale. Second, the gap *widens* in favor of continuous models as the number of steps decreases [15, 23, 24], which is the regime where the factorization error of discrete models becomes larger. Third, some continuous models report their best perplexity at slightly lower entropy than baselines, whereas comparisons at matched entropy or along generative frontiers are necessary for drawing reliable conclusions.

7.1.2 Evaluation Metrics

Unconditional generation is usually evaluated by the **generative perplexity** (gPPL) of samples under a reference AR model p_{ref}, typically GPT-2 Large [1], which is computed as:

$$\mathrm{gPPL}(\theta) = \exp\left(\mathbb{E}_{x\sim p_\theta}\left[-\frac{1}{L-1}\sum_{\ell=2}^{L}\log p_{\mathrm{ref}}(x^\ell \mid x^{<\ell})\right]\right)$$

paired with the average unigram entropy of the samples to detect degenerate, repetitive text [3, 216]:

$$\mathrm{Entropy}(\theta) = \mathbb{E}_{\boldsymbol{x}\sim p_\theta}\left[-\textstyle\sum_{v\in\mathcal{V}}\hat{q}_{\boldsymbol{x}}(v)\log\hat{q}_{\boldsymbol{x}}(v)\right] \tag{33}$$

where $\hat{q}_{\boldsymbol{x}}(v)$ is the fraction of token v in the sequence $\boldsymbol{x}$. Likelihood-based evaluation is less straightforward for continuous models than for AR, since it requires an ODE-based or variational bound [12, 16, 18].

Recent work shows that these pointwise metrics fail to provide a complete picture of the tradeoff between

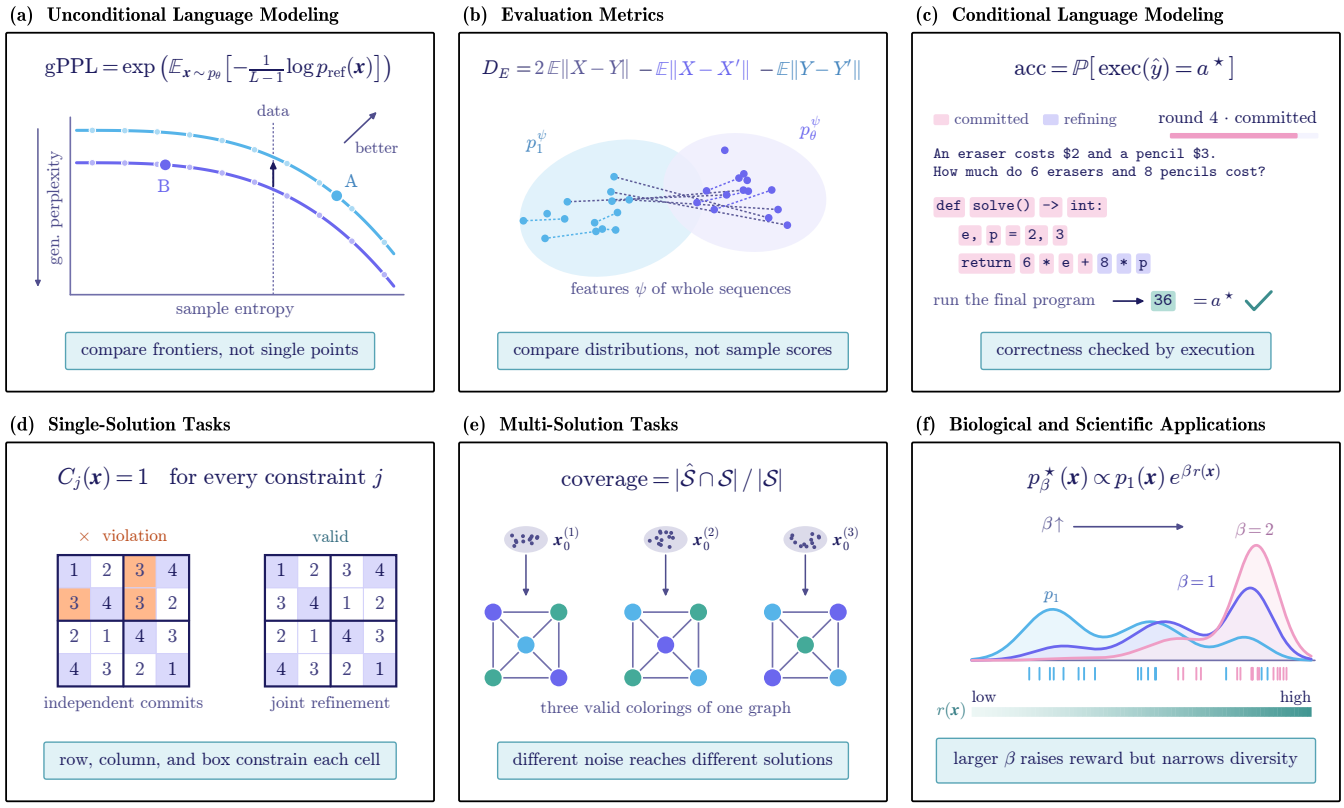


Fig. 6: Evaluation settings for continuous language models. (a) Each frontier traces one method across a sweep of sampling hyperparameters, and each dot is one setting. Point B has lower perplexity than point A, but A has higher entropy, and its frontier is better at the entropy of the data. **(b)** Distributional metrics compare generated and reference samples in a feature space. The energy distance combines distances between the two sets with distances within each set, and the dashed lines show one example pair per term. **(c)** Small-scale tasks such as TinyGSM execute each generated program and check its output, which tests conditional language reasoning. **(d)** Puzzles such as Sudoku expose the factorization error, since cells committed independently can violate row, column, and box constraints. **(e)** Multi-solution tasks reward models that sample diverse and accurate solutions that cover the set of valid solutions. **(f)** Biological design often targets the base distribution p_1 tilted by a reward r. As β grows, samples (ticks) concentrate where the reward is high but cover fewer modes, so evaluation should report reward together with diversity.

likelihood and token diversity. Zheng et al. [217] first observed that generative perplexity can be lowered simply by reducing token diversity. They showed that numerical imprecision in categorical sampling implicitly lowers the sampling temperature, which may have unfairly favored masked diffusion models in earlier comparisons. Pynadath et al. [218] make this tradeoff explicit by showing that generative perplexity and entropy are two competing components of the KL divergence to a reference distribution. Lowering the sampling temperature or using a more mode-seeking sampler trades entropy for perplexity, so a single (gPPL, entropy) point gives an inconclusive comparison between methods. They therefore propose to compare methods by their *generative frontiers* [59], the curves traced as entropy is varied through sampling hyperparameters such as the decoding temperature, the number of function evaluations (NFE), and the amount of injected noise. They further argue for OpenWebText as the standard benchmark over LM1B. OpenWebText is a large and lightly curated corpus that openly reproduces the collection procedure of WebText, the training data of GPT-2, making GPT-2 Large a well-matched reference model for measuring gPPL. LM1B, in contrast, was designed for sentence-level likelihood evaluation and is shuffled at the sentence level, so it cannot reveal whether a model produces coherent text beyond a single sentence.

Franca and Tong [219] go further and show that gPPL can be hacked while preserving the entropy close to that of the data. Specifically, training-free samplers can produce incoherent text with state-of-the-art gPPL by favoring high-frequency tokens and sampling periodic repeats, templates, or copies following each prefix. This artificially produces low gPPL because the metric measures predictability under the scoring model rather than grammaticality or coherence. To overcome these failure modes, they recommend reporting diversity diagnostics together with *distributional metrics*, which directly compare the distribution of generated samples p_θ with the data distribution p_1. **MAUVE** [220] embeds both sets of samples with a reference language model, quantizes the embeddings into clusters, and compares the resulting histograms $\hat{p}_\theta$ and $\hat{p}_1$ through their mixtures $p_\lambda = \lambda\hat{p}_\theta + (1-\lambda)\hat{p}_1$ with a scaling constant $c > 0$,

$$\begin{aligned} &\mathrm{MAUVE} = \mathrm{AUC}(\mathcal{C}), \\ &\mathcal{C} = \left\{ \left(e^{-c\, D_{\mathrm{KL}}(\hat{p}_1 \| p_\lambda)},\ e^{-c\, D_{\mathrm{KL}}(\hat{p}_\theta \| p_\lambda)} \right) : \lambda \in (0,1) \right\}. \end{aligned} \tag{34}$$

The two coordinates measure how much of the real distri-

bution the model misses and how much of the generated distribution falls outside the real one, so the area under the curve rewards both quality and coverage. The **gradient moment** (GM) distance [221] measures whether generated and real samples would push a fixed reference language model with parameters ϕ in the same training direction,

$$\mathrm{GM} = \left\| \mathbb{E}_{\boldsymbol{x}\sim p_\theta} \nabla_\phi \log p_{\mathrm{ref}}(\boldsymbol{x}) - \mathbb{E}_{\boldsymbol{x}\sim p_1} \nabla_\phi \log p_{\mathrm{ref}}(\boldsymbol{x}) \right\|_2^2,$$

which is a squared MMD with the Fisher score $x \mapsto \nabla_\phi \log p_{\mathrm{ref}}(\boldsymbol{x})$ as its feature map. It reaches zero, its minimum, if and only if these expected scores are equal under p_θ and p_1. The **energy distance** [222] D_E maps generated and real sequences to features with a projection $\psi : \mathcal{V}^L \to \mathbb{R}^d$ and compares the two projected distributions as

$$\begin{aligned} D_E(p_\theta^\psi, p_1^\psi) = 2\mathbb{E}_{X\sim p_\theta^\psi, Y\sim p_1^\psi}[\|X-Y\|] \\ - \mathbb{E}_{X,X'\sim p_\theta^\psi}[\|X-X'\|] - \mathbb{E}_{Y,Y'\sim p_1^\psi}[\|Y-Y'\|] \end{aligned} \tag{35}$$

where $D_E \geq 0$ and $D_E = 0$ if and only if the feature-projected distributions are equal, $p_\theta^\psi = p_1^\psi$. Empirically, naive samplers perform poorly under these distributional metrics. These metrics expose the demonstrated failure modes of gPPL, although finite-sample estimates and chosen representations do not guarantee identification of the complete text distribution, and they reveal larger differences across NFE than gPPL does [219].

SOL [223] is a distributional metric that avoids the clustering step of MAUVE. Each sequence x is represented by the empirical measure of its contextual hidden states under a fixed reference model, $T(x) = \frac{1}{|x|}\sum_\ell \delta_{h_{x_{\leq \ell}}}$, so that sequences of different lengths can be compared directly. Comparing two corpora exactly would require a Wasserstein distance between distributions of point clouds, so SOL uses random one-dimensional projections, which is known as slicing. It projects each point cloud onto a random direction ξ and then projects the quantile function of the result onto a random Gaussian-process direction g, and it averages the resulting one-dimensional Wasserstein distances,

$$\begin{aligned} &\mathrm{DSW}^2(\boldsymbol{\mu}, \boldsymbol{\nu}) = \mathop{\mathbb{E}}_{\xi\sim\mathcal{U}(\mathbb{S}^{d-1}), g\sim\gamma}\Big[W_2^2\big((M_{\xi,g})_\#\boldsymbol{\mu},\ (M_{\xi,g})_\#\boldsymbol{\nu}\big)\Big], \\ &M_{\xi,g} : \mathcal{P}_2(\mathbb{R}^d) \to \mathbb{R}, \quad \mu \mapsto \langle Q_{\xi_\#\mu},\ g\rangle_{L^2}, \\ &(M_{\xi,g})_\# : \mathcal{P}_2(\mathcal{P}_2(\mathbb{R}^d)) \to \mathcal{P}_2(\mathbb{R}), \quad \boldsymbol{\mu} \mapsto (M_{\xi,g})_\#\boldsymbol{\mu}, \end{aligned}$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\nu}$ are the distributions over point clouds of the data and the model, respectively, μ is a single point cloud, Q denotes a quantile function, and γ is the Gaussian-process distribution from which g is drawn. Each one-dimensional distance reduces to sorting, so the metric is cheap once hidden states are extracted. Under the assumptions of its separation theorem, including an injective last-token representation and a full-support Gaussian measure, SOL distinguishes the relevant empirical text distributions. Finite random-slice approximations do not automatically inherit exact separation. It also exposes the failure modes discussed above. For example, replacing samples with copies of a single sequence keeps the token entropy near the level of data and lowers gPPL, yet sharply increases SOL, and two continuous models that differ widely in gPPL turn out to be much closer under SOL.

Together, these results suggest that gPPL comparisons should be accompanied by entropy, generative frontiers, distributional metrics, and across-sample diversity checks.

7.2 Conditional Language Modeling

Beyond unconditional generation, continuous models are increasingly evaluated on conditional language tasks. Early embedding models already targeted sequence-to-sequence problems. **DiffuSeq** [50] conditions on the source through partial noising and evaluates on paraphrasing, question generation, and dialogue, while **SeqDiffuSeq** [68] and **GENIE** [69] encode the source with a separate encoder and extend the evaluation to machine translation and summarization, respectively. **ELF** [17] returns to translation and summarization with a different form of conditioning, classifier-free guidance on the input, and is competitive on both. The recent focus, however, has shifted to reasoning in natural language, where answers can be checked automatically.

7.2.1 Tasks and Evaluation for Small Models

For academic-scale models, the **TinyGSM** [224] benchmark has become a standard setting. TinyGSM is a synthetic dataset of grade-school math problems paired with Python solutions. In this setting, a small DiT of roughly the size used for OpenWebText experiments is trained from scratch on the problems and programs, with noise applied only to the program and not to the question. The model is evaluated zero-shot on the **GSM8K** test set [225] by executing one generated program per problem and checking its output against the answer. This protocol was introduced to masked diffusion models by Kim et al. [226] and has since been adopted by several continuous language models.

We argue that TinyGSM is a particularly good setting for comparing continuous and discrete language models at small scale because it avoids the main problems of gPPL and entropy. First, correctness is verified by execution, so repetition or templates cannot inflate the metric. Second, a single correct answer per problem removes the need to match entropy across methods, while the program must also be syntactically valid and semantically correct, which tests the joint dependencies between tokens that parallel decoding struggles with. The setting is also relatively cheap to evaluate, allowing hyperparameter sweeps over the number of sampling steps, temperature, and refinement budget. Hybrid models such as **CANDI** and standard one-hot flows like **FLM** perform poorly, whereas **S-FLM** closes the gap to discrete diffusion and **SDM** outperforms them [15, 58, 59, 103]. Furthermore, **FMLM+**, **DBTM**, and ***a*FMLM** reach strong accuracy with one or a few function evaluations [24, 137, 186]. TinyGSM thus separates design choices that look similar in the unconditional setting.

7.2.2 Tasks and Evaluation for Large Models

In scaled settings, larger continuous models are evaluated on the standard downstream benchmarks used for large AR and discrete diffusion language models, after supervised fine-tuning. These benchmarks cover math, such as GSM8K [225] and MATH-500 [227], coding, such as HumanEval [2, 228] and MBPP [228, 229], knowledge-based reasoning, such as MMLU [230] and GPQA [231], and

instruction following, such as IFEval [232]. **ELF-REG** [78] extends ELF to math and code. It adds *representation alignment* (REPA) [233, 234], which aligns intermediate features of the denoiser with the clean representations of a frozen AR teacher, and *joint denoising* of the sequence and a global latent (REG) [235]. It also uses the early stopping decoder described in Section 4.5. With these additions, ELF-REG is competitive with continuous diffusion, masked diffusion, and small AR models on the math and code benchmarks. **MLFM** [139] adapts pretrained masked diffusion models into flows and is among the first flow language models evaluated on instruction following. More recently, **Sigma** [19] and **Lafa** [20] adopt pretrained AR models and evaluate across all of these categories. These systems report competitive performance in their respective evaluation settings. Continuous-state generation permits revision of tentative predictions, although these comparisons do not isolate revision as the cause of the performance gains.

7.3 Reasoning for Puzzle Solving

The iterative, globally coupled refinement of continuous flows makes them natural candidates for structured reasoning puzzles, where many interdependent decisions must be mutually consistent, and the answer can be verified exactly.

7.3.1 Single-Solution Tasks

Single-solution benchmarks for continuous models are typically puzzles scored by exact-match accuracy, and together they test increasingly abstract forms of consistency. **Sudoku** tests local constraints, since every cell is constrained by its row, column, and box [236]. Zebra puzzles, or logic grid puzzles, require deducing a unique assignment of attributes to entities from a list of relational clues, so the constraints change from one instance to the next. **Mazes** test long-range consistency where the model must output an optimal path through a large grid [236]. **ARC-AGI-1** [237] and **ARC-AGI-2** [238] are more difficult, as each instance offers only a few input and output grids, and the model must infer an abstract transformation from them and apply it to a new input.

Sudoku is the most common of these benchmarks for continuous language models [58, 103, 123, 136], usually evaluated at four difficulty levels. It directly tests the factorization error, since every cell is constrained by its row, column, and box, and committing several cells independently from their marginals will frequently produce violations. Many-step flows reach near-perfect validity, and one-step flow and transport maps already solve a nontrivial fraction of puzzles [15, 24]. Asynchronous decoding (Section 5.5.2) further improves accuracy over standard decoding [24, 137, 186]. Bai et al. [239] also showed that under matched numbers of denoising steps, FLMs solve Sudoku and mazes more accurately than discrete diffusion, as their continuous states keep evidence for several candidates until the final decoding. Random constraint satisfaction problems such as random k-SAT give a more controlled setting, because the ideal local denoiser is known from theory and the sampler can therefore be evaluated separately from the learned denoiser. In this setting, Bhatt et al. [240] showed that continuous diffusion outperforms masked diffusion.

A second line of work uses continuous flows as recurrent reasoners rather than as language models, and evaluates them across more difficult benchmarks such as Sudoku-Extreme, Maze-Unique, and ARC-AGI. **FRM** [122] applies self-conditioning (Section 3.4) recursively and trains the model on states that are produced by its own recursion. **Looped Flows** [123] instead train recurrent updates with local denoising objectives and shared noise, so that hidden states carry the computation forward, even though gradients cover only a few iterations. At inference, they integrate a probability flow velocity coupled with the recurrent state, and a finer time grid gives more computation to harder problems. Both models improve on recurrent reasoners such as the Hierarchical Reasoning Model [236] and the Tiny Recursive Model [241]. In particular, FRM nearly saturates Sudoku, Zebra puzzles, and mazes with a small fraction of the inference compute of these baselines, while Looped Flows also outperform them on both ARC-AGI benchmarks.

7.3.2 Multi-Solution Tasks

Many reasoning problems have more than one correct answer, and a deterministic predictor that is trained on one reference solution per input tends to collapse onto a single answer or to average several answers into an invalid one. To expose this failure, Baek et al. [242] introduce multi-solution benchmarks and show that deterministic recursive reasoners lose accuracy as the number of valid solutions grows. In **N-Queens**, queens are hidden from a partially filled board and must be placed so that no two attack each other. In **Graph Coloring**, every vertex of a random three-colorable graph must receive one of three colors so that no edge joins two vertices of the same color. For N-Queens, evaluation reports first-sample validity and solution coverage. For Graph Coloring, it reports the number of conflicts in the most frequently sampled coloring and solution coverage.

Together, accuracy and coverage distinguish a mode-seeking model, which can be accurate yet cover only a single solution, from a diverse sampler, which can cover many solutions while committing more errors. Continuous generative models are well placed to do both, because different initial noise samples are transported to different modes of the solution set. Accordingly, Looped Flows report improvements on task-specific correctness and coverage measures, with particularly strong gains on larger multi-solution instances. While stochastic integration generally improves coverage, ODE integration largely retains the diversity, which indicates that the diversity comes from probability transport through the learned flow rather than from injected noise alone.

7.4 Biological and Scientific Applications

Biological sequences such as proteins, peptides, and nucleic acids are discrete strings over small vocabularies, so methods developed for text can be adapted to biological design. We review the tasks and datasets used in this setting, the role of conditional generation and reward guidance, and the metrics used to evaluate designed sequences.

7.4.1 Tasks and Datasets

Discrete and continuous diffusion have been shown to often outperform comparable AR models on biological sequence design [243], with more advantages than in natural

language. This can be partly explained by the fact that language is predominantly causal, since text is read and written from left to right. In contrast, biological sequences have stronger bidirectional and long-range dependencies, since two tokens that are far apart in the sequence can be close in three-dimensional space. This has motivated new approaches to protein, peptide, DNA, and RNA sequence design and to small-molecule generation, including discrete diffusion [244, 245], discrete flows [8, 246], continuous flows over probabilities [55–57] or over protein language model encodings [247, 248], and flow maps [22].

7.4.2 Conditional Generation and Reward Guidance

Most biological tasks require reward optimization or constrained generation, which demands precise control of the generative trajectory while capturing correlations across the full sequence. While various methods for reward guidance and fine-tuning in discrete diffusion demonstrate promising results for biological design tasks [41, 245, 246, 249, 250], they retain the factorization error of discrete diffusion and require many generation steps. Due to the non-differentiable state space, discrete-space models rely on reweighting [250] or first-order approximations [41] for reward guidance, whereas continuous-space models can steer with the gradient of the reward function, letting the reward-tilted dynamics guide the trajectory to potentially new regions of the state space. **Dirichlet FM** [55] adapts both classifier-free and classifier-based guidance to guide promoter DNA design toward higher regulatory activity, as predicted by Chen et al. [251]. Similarly, **Gumbel-Softmax FM** [56] introduces STGFlow, which samples discrete sequences at intermediate timesteps and applies the straight-through estimator to guide peptide generation toward target binding affinity. Classifier-free guidance also applies in latent spaces, as **CHASE** [252] guides a flow over compressed protein language model embeddings toward high-fitness variants.

7.4.3 Evaluation Metrics

Evaluation of biological and molecular generation commonly considers validity, novelty, uniqueness, and synthesizability [253]. For molecules, *validity* measures whether generated strings can be decoded into valid chemical structures and motifs [254]. For proteins and other biological sequences, it can instead refer to sequence, structural, or functional validity. *Novelty* measures how distinct generated samples are from the training data, while *uniqueness* measures the fraction of distinct samples within the generated set. For biological sequences, novelty should also consider sequence similarity or homology, since a sequence that has no exact match in the training set can still be nearly identical to one of them. *Synthesizability* assesses the practical feasibility of producing the generated sequences or molecules.

These general metrics are complemented by task-specific measures of biological quality and function. Protein design can be evaluated using predicted structural confidence and sequence-structure consistency [244, 255, 256], peptide design using predicted binding affinity, predicted properties, and docking scores [56, 257], small-molecule design using **Fréchet ChemNet Distance** (FCD) [258], and DNA/RNA design using predicted regulatory activity [259] or **Fréchet Biological Distance** (FBD) [55]. Computational predictions should be validated experimentally, since improvements in a predictor's score do not necessarily translate into improved biological function. For reward-guided generation, evaluation should also report the tradeoff between reward improvement and sample diversity or constraint satisfaction as the guidance strength or regularization weight varies.

8 Challenges and Future Directions

Recent advances have made continuous language models viable at larger scales and in the few-step regime. Continued progress would turn these gains into efficient training and deployment that retain the benefits in control and joint generation. We review related challenges and future directions.

8.1 Scalable Language Representations

A useful representation of language for continuous modeling must be inexpensive to generate and reliably decodable. Low-dimensional embeddings reduce the size of the evolving state compared with one-hot token encodings, but the best way to obtain them remains unsettled. Pretrained encoders provide linguistic structure, although their representations may require adaptation [17, 84, 89]. Learning representations jointly with the generator allows them to adapt to the generative task, but requires balancing reconstruction and generation objectives to avoid collapse [11, 53, 86]. Recent work in image generation shows that this balance can be managed effectively at scale, with representations and diffusion models learned or fine-tuned together [260–263]. These results offer useful starting points for making joint representation learning reliable and scalable for language, allowing the representation to improve with the generator. Hierarchical representations offer another promising direction. Ollu and Komodakis [65] jointly denoise token representations and coarser semantic cluster codes, demonstrating that modeling multiple levels of abstraction can improve generation with little additional overhead.

8.2 Scaling Pretraining and Few-Step Distillation

Continuous language models around the 10B-parameter scale have largely been built by adapting pretrained AR models [19, 20, 128], while the token-level and blockwise continuous models surveyed here have been trained from scratch at up to 3B parameters [18, 71, 86, 206]. Adaptation provides a pragmatic route to strong models by reusing linguistic knowledge and hardware-efficient implementations, and enables practical uses, especially speculative decoding [20, 165]. It nevertheless starts from representations and inductive biases shaped by next-token prediction, which may be difficult to revise through adaptation. A central challenge is to understand distinctive capabilities that large-scale continuous pretraining can develop by learning to generate and refine many tokens jointly from the outset. Better data efficiency is one possibility, as suggested by the gains from repeated data observed in masked diffusion [264, 265].

Turning stronger continuous models into fast generators requires preserving their quality in one or a few sampling steps. Guidance, self-conditioning, and stochastic refinement can substantially improve iterative generation, but their benefits must be retained as repeated computation is

replaced by learned updates. Distilling guided predictions and self-conditioning feedback offers ways to address this problem [17, 77], while learning flow maps of SDEs could extend distillation to stochastic samplers [266, 267]. Bringing these methods together at scale could narrow the gap between few-step generators and their iterative counterparts. This further motivates studying optimal resource allocation between pretraining and few-step distillation, and whether unified training via self-distillation can be more efficient.

8.3 Post-training

Post-training is central to instruction following, reasoning, and alignment of language models [268, 269]. Supervised fine-tuning can reuse continuous pretraining objectives, but common reinforcement learning (RL) methods rely on likelihood ratios that are cheap for AR models and costly for continuous generators [269, 270]. One route is to make likelihood evaluation efficient by learning it alongside sampling [271, 272] or constructing flow maps with tractable continuous-state densities [273], although obtaining the probability of a discrete decoded text sequence may still require marginalization through the decoder. Another route uses stochastic samplers with tractable transition densities and likelihood ratios [274, 275], avoiding the need to evaluate final-output marginal likelihoods. Reward objectives can also be differentiated directly through flow maps [276]. Han et al. [277] apply this idea to a flow language model, using a critic on intermediate token embeddings.

8.4 Multimodality

Continuous representations allow text and other modalities to be generated through a shared process [79, 278, 279], unlike hybrids that pair AR text with diffusion over images [280]. This creates an opportunity to coordinate decisions across modalities during refinement. Designing representations and schedules can pose new challenges. In text, for example, token identities can change rapidly within a narrow range of noise levels, unlike in images and audio. A unified model should allocate each modality a suitable refinement budget while maintaining coherence between their outputs. Few-step distillation makes this coordination especially important, since a small number of updates must resolve both the content in each modality and dependencies between them.

8.5 Safety

As training methods improve and language models become more capable, safety becomes an increasingly important concern [281]. Continuous generation can pose a challenge for monitoring, as its evolving state is not directly readable as text, making approaches based on linguistic reasoning traces less straightforward [282]. Likewise, generating many tokens in a few updates can leave fewer opportunities to inspect or interrupt a developing response. One possibility is to use the decoder or a trained probe to turn intermediate states or hidden features into tentative text, providing previews on which safety monitors could operate to inform interventions during refinement. Tools for interpreting, monitoring, and constraining continuous trajectories will become increasingly important as these models scale.

9 Conclusion

This survey reviewed continuous diffusion and flow language models, which use continuous representations to support joint generation across tokens, differentiable control, and learned few-step sampling. We organized their design and training choices, connected them to inference and distillation, and examined recent progress in scaling, applications, and evaluation. The field has advanced from small-scale demonstrations to billion-parameter models, establishing continuous generation as a viable approach to language modeling. Continued progress hinges on learning suitable representations, scaling continuous pretraining and distillation, and developing post-training and monitoring methods that remain effective in the few-step regime. These directions could turn the flexibility of continuous generation into practical gains in capability, efficiency, and control.

References

[1] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, pages 130136–130184, 2024.

[4] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis Titsias. Simplified and generalized masked diffusion for discrete data. *Advances in Neural Information Processing Systems*, pages 103131–103167, 2024.

[5] Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. *International Conference on Learning Representations*, pages 64972–65009, 2025.

[6] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34: 17981–17993, 2021.

[7] Yair Schiff, Subham Sekhar Sahoo, Hao Phung, Guanghan Wang, Sam Boshar, Hugo Dalla-torre, Bernardo P. de Almeida, Alexander Rush, Thomas Pierrot, and Volodymyr Kuleshov. Simple guidance mechanisms for discrete diffusion models. In *International Conference on Learning Representations*, pages 43776–43821, 2025.

[8] Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows

with applications to protein co-design. In *International Conference on Machine Learning*, pages 5453–5512, 2024.

[9] Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky TQ Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching. *Advances in Neural Information Processing Systems*, 37:133345–133385, 2024.

[10] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. *Advances in neural information processing systems*, 35:4328–4343, 2022.

[11] Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, et al. Continuous diffusion for categorical data. *arXiv preprint arXiv:2211.15089*, 2022.

[12] Ishaan Gulrajani and Tatsunori B Hashimoto. Likelihood-based diffusion language models. In *Advances in Neural Information Processing Systems*, pages 16693–16715, 2023.

[13] Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. Scaling up masked diffusion models on text. In *International Conference on Learning Representations*, volume 2025, pages 82974–82997, 2025.

[14] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *Advances in Neural Information Processing Systems*, pages 50608–50646, 2025.

[15] Chanhyuk Lee, Jaehoon Yoo, Manan Agarwal, Sheel Shah, Jerry Huang, Aditi Raghunathan, Seunghoon Hong, Nicholas M Boffi, and Jinwoo Kim. Flow map language models: One-step language modeling via continuous denoising. *arXiv preprint arXiv:2602.16813*, 2026.

[16] Yuxin Chen, Chumeng Liang, Hangke Sui, Ruihan Guo, Chaoran Cheng, Jiaxuan You, and Ge Liu. Langflow: Continuous diffusion rivals discrete in language modeling. *arXiv preprint arXiv:2604.11748*, 2026.

[17] Keya Hu, Linlu Qiu, Yiyang Lu, Hanhong Zhao, Tianhong Li, Yoon Kim, Jacob Andreas, and Kaiming He. Elf: Embedded language flows. *arXiv preprint arXiv:2605.10938*, 2026.

[18] Oscar Davis, Anastasiia Filippova, Pierre Ablin, Victor Turrisi, Amitis Shidani, Marco Cuturi, and Louis Béthune. Scaling categorical flow maps. *arXiv preprint arXiv:2605.07820*, 2026.

[19] Zhihan Yang, Wei Guo, Jean-Marie Lemercier, Simon Welker, Yonggan Fu, Mohammad Mahdi Kamani, Sajad Norouzi, Julius Berner, Tomas Geffner, Karsten Kreis, Yongxin Chen, Molei Tao, John Thickstun, Pavlo Molchanov, Ante Jukić, Arash Vahdat, and Morteza Mardani. Large language continuous diffusion models. *arXiv preprint arXiv:2610.02665*, 2026.

[20] Arnav Chavan, Chanhyuk Lee, Udbhav Bamba, Jaehoon Yoo, Seunghoon Hong, Aditi Raghunathan, and Jinwoo Kim. Large flow language models. *arXiv preprint arXiv:2610.28559*, 2026.

[21] Nicholas M Boffi, Michael S Albergo, and Eric Vanden-Eijnden. How to build a consistency model: Learning flow maps via self-distillation. *Advances in Neural Information Processing Systems*, pages 33346–33382, 2025.

[22] Daan Roos, Oscar Davis, Floor Eijkelboom, Michael Bronstein, Max Welling, Ismail Ilkan Ceylan, Luca Ambrogioni, and Jan-Willem van de Meent. Categorical flow maps. *International Conference on Machine Learning*, pages 105618–105642, 2026.

[23] Peter Potaptchik, Jason Yim, Adhi Saravanan, Peter Holderrieth, Eric Vanden-Eijnden, and Michael S Albergo. Discrete flow maps. *arXiv preprint arXiv:2604.09784*, 2026.

[24] Sophia Tang and Shiyi Wang. Discrete beckmann transport models for one-step language modeling and reasoning. *arXiv preprint arXiv:2609.15903*, 2026.

[25] Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. A survey on diffusion language models. *arXiv preprint arXiv:2508.10875*, 2025.

[26] Sander Dieleman. Diffusion language models, 2023. URL https://benanne.github.io/2023/01/09/diffusion-language.html.

[27] Sander Dieleman. Continuous diffusion language models, 2026. URL https://sander.ai/2026/08/24/continuous-dlms.html.

[28] Andrew Campbell, Joe Benton, Valentin De Bortoli, Thomas Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. *Advances in Neural Information Processing Systems*, 35:28266–28279, 2022.

[29] Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. *Advances in Neural Information Processing Systems*, 38:147282–147339, 2025.

[30] Justin Deschenaux and Caglar Gulcehre. Beyond autoregression: Fast LLMs via self-distillation through time. In *International Conference on Learning Representations*, pages 5007–5045, 2025.

[31] Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, Guanghan Wang, Justin T Chiu, and Volodymyr Kuleshov. The diffusion duality. In *International Conference on Machine Learning*, pages 52584–52619, 2025.

[32] Satoshi Hayakawa, Yuhta Takida, Masaaki Imaizumi, Hiromi Wakaki, and Yuki Mitsufuji. Distillation of discrete diffusion through dimensional correlations. In *International Conference on Machine Learning*, pages 22259–22297, 2025.

[33] Jean-Marie Lemercier, Tomas Geffner, Karsten Kreis, Morteza Mardani, Arash Vahdat, and Ante Jukić. DiLaDiff: Distilled latent-augmented diffusion for language modeling. *arXiv preprint arXiv:2605.23605*, 2026.

[34] Tianyu Xie, Shuchen Xue, Zijin Feng, Tianyang Hu, Jiacheng Sun, Zhenguo Li, and Cheng Zhang. Variational autoencoding discrete diffusion with enhanced dimensional correlations modeling. In *International Conference on Learning Representations*, pages 39109–39132, 2026.

[35] Minkai Xu, Tomas Geffner, Karsten Kreis, Weili Nie, Yilun Xu, Jure Leskovec, Stefano Ermon, and Arash Vahdat. Energy-based diffusion language models for text generation. In *International Conference on Learning*

Representations, pages 33769–33789, 2025.

[36] Anji Liu, Oliver Broadrick, Mathias Niepert, and Guy Van den Broeck. Discrete copula diffusion. In *International Conference on Learning Representations*, pages 88953–88979, 2025.

[37] Jaehoon Yoo, Wonjung Kim, and Seunghoon Hong. ReDi: Rectified discrete flow. In *Advances in Neural Information Processing Systems*, pages 81651–81683, 2025.

[38] Jaehoon Yoo, Wonjung Kim, Chanhyuk Lee, and Seunghoon Hong. Infinite mask diffusion for few-step distillation. In *International Conference on Machine Learning*, pages 149532–149558, 2026.

[39] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[40] Xiaochuang Han, Sachin Kumar, and Yulia Tsvetkov. Ssd-lm: Semi-autoregressive simplex-based diffusion language model for text generation and modular control. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11575–11596, 2023.

[41] Hunter Nisonoff, Junhao Xiong, Stephan Allenspach, and Jennifer Listgarten. Unlocking guidance for discrete state-space diffusion and flow models. In *International Conference on Learning Representations*, volume 2025, pages 36052–36106, 2025.

[42] Xiner Li, Yulai Zhao, Chenyu Wang, Gabriele Scalia, Gokcen Eraslan, Surag Nair, Tommaso Biancalani, Shuiwang Ji, Aviv Regev, Sergey Levine, and Masatoshi Uehara. Derivative-free guidance in continuous and discrete diffusion models with soft value-based decoding. In *Advances in Neural Information Processing Systems*, pages 95507–95545, 2025.

[43] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[44] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

[45] Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.

[46] Michael S Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations*, 2023.

[47] Floor Eijkelboom, Grigory Bartosh, Christian A Naesseth, Max Welling, and Jan-Willem van de Meent. Variational flow matching for graph generation. *Advances in Neural Information Processing Systems*, 37: 11735–11764, 2024.

[48] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020.

[49] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[50] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and LingPeng Kong. Diffuseq: Sequence to sequence text generation with diffusion models. In *International Conference on Learning Representations*, 2023.

[51] Robin Strudel, Corentin Tallec, Florent Altché, Yilun Du, Yaroslav Ganin, Arthur Mensch, Will Grathwohl, Nikolay Savinov, Sander Dieleman, Laurent Sifre, et al. Self-conditioned embedding diffusion for text generation. *arXiv preprint arXiv:2211.04236*, 2022.

[52] Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q Weinberger. Latent diffusion for language generation. *Advances in Neural Information Processing Systems*, 36:56998–57025, 2023.

[53] Viacheslav Meshchaninov, Alexander Shabalin, Egor Chimbulatov, Nikita Gushchin, Ilya Koziev, Alexander Korotin, and Dmitry Vetrov. How to train your latent diffusion language model jointly with the latent space. *arXiv preprint arXiv:2605.07933*, 2026.

[54] Rabeeh Karimi Mahabadi, Hamish Ivison, Jaesung Tae, James Henderson, Iz Beltagy, Matthew E Peters, and Arman Cohan. Tess: Text-to-text self-conditioned simplex diffusion. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2347–2361, 2024.

[55] Hannes Stark, Bowen Jing, Chenyu Wang, Gabriele Corso, Bonnie Berger, Regina Barzilay, and Tommi Jaakkola. Dirichlet flow matching with applications to DNA sequence design. In *International Conference on Machine Learning*, pages 46495–46513, 2024.

[56] Sophia Tang, Yinuo Zhang, Alexander Tong, and Pranam Chatterjee. Gumbel-softmax flow matching with straight-through guidance for controllable biological sequence generation. *arXiv preprint arXiv:2503.17361*, 2025.

[57] Oscar Davis, Samuel Kessler, Mircea Petrache, İsmail İlkan Ceylan, Michael Bronstein, and Avishek Joey Bose. Fisher flow matching for generative modeling over discrete data. In *Advances in Neural Information Processing Systems*, pages 139054–139084, 2024.

[58] Justin Deschenaux and Caglar Gulcehre. Language modeling with hyperspherical flows. *arXiv preprint arXiv:2605.11125*, 2026.

[59] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. CANDI: Hybrid discrete-continuous diffusion models. In *International Conference on Machine Learning*, pages 100176–100208, 2026.

[60] Huangjie Zheng, Shansan Gong, Ruixiang Zhang, Tianrong Chen, Jiatao Gu, Mingyuan Zhou, Navdeep Jaitly, and Yizhe Zhang. Continuously augmented discrete diffusion model for categorical generative modeling. In *International Conference on Learning Representations*, pages 34212–34242, 2026.

[61] Cai Zhou, Chenxiao Yang, Yi Hu, Chenyu Wang, Chubin Zhang, Muhan Zhang, Lester Mackey, Tommi Jaakkola, Stephen Bates, and Dinghuai Zhang. Coevolutionary continuous discrete diffusion: Make your diffusion language model a latent reasoner. In *International Conference on Machine Learning*, pages 166143–166171, 2026.

[62] Ting Chen, Ruixiang Zhang, and Geoffrey Hinton.

Analog bits: Generating discrete data using diffusion models with self-conditioning. In *International Conference on Learning Representations*, 2023.

[63] Georgios Batzolis, Mark Girolami, and Luca Ambrogioni. Cobit: Language modeling with bitstream diffusion. *arXiv preprint arXiv:2605.07013*, 2026.

[64] Shaobin Zhuang, Yuang Ai, Jiaming Han, Xiaohui Li, Huaibo Huang, Xiangyu Yue, Xuefeng Hu, Kun Xu, Yali Wang, and Hao Chen. Bitlm: Unlocking multi-token language generation with bitwise continuous diffusion. *arXiv preprint arXiv:2605.11577*, 2026.

[65] Mathias Ollu and Nikos Komodakis. Denoising hierarchical representations: Joint continuous diffusion for language modeling. *arXiv preprint arXiv:2610.08738*, 2026.

[66] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[67] Alexander Shabalin, Viacheslav Meshchaninov, and Dmitry Vetrov. Smoothie: Smoothing diffusion on token embeddings for text generation. In *International Conference on Machine Learning*, pages 109429–109446, 2026.

[68] Hongyi Yuan, Zheng Yuan, Chuanqi Tan, Fei Huang, and Songfang Huang. Seqdiffuseq: Text diffusion with encoder-decoder transformers. *arXiv preprint arXiv:2212.10325*, 2022.

[69] Zhenghao Lin, Yeyun Gong, Yelong Shen, Tong Wu, Zhihao Fan, Chen Lin, Nan Duan, and Weizhu Chen. Text generation with diffusion language models: A pre-training approach with continuous paragraph denoise. In *International Conference on Machine Learning*, pages 21051–21064. PMLR, 2023.

[70] Zhujin Gao, Junliang Guo, Xu Tan, Yongxin Zhu, Fang Zhang, Jiang Bian, and Linli Xu. Empowering diffusion models on the embedding space for text generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4664–4683, 2024.

[71] Zhihan Yang, Wei Guo, Shuibai Zhang, Subham Sekhar Sahoo, Yongxin Chen, Arash Vahdat, Morteza Mardani, and John Thickstun. Continuous diffusion scales competitively with discrete diffusion for language. In *Advances in Neural Information Processing Systems*, 2026.

[72] Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. Learned in translation: Contextualized word vectors. *Advances in neural information processing systems*, 30:6294–6305, 2017.

[73] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, 2018.

[74] Alexander Shabalin, Viacheslav Meshchaninov, Egor Chimbulatov, Vladislav Lapikov, Roman Kim, Grigory Bartosh, Dmitry Molchanov, Sergey Markov, and Dmitry Vetrov. Tencdm: Understanding the properties of the diffusion model in the space of language model encodings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25110–25118, 2025.

[75] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[76] Guangfu Guo, Xiaoqian Lu, Linsey Pang, Weiran Yao, Haolin Chen, Kunpeng Liu, and Long Cheng. Deltaflow: Noise-adaptive bidirectional gated delta networks for embedded language flows. *arXiv preprint arXiv:2608.01240*, 2026.

[77] Jaehoon Yoo, Wonjung Kim, Floor Eijkelboom, Chanhyuk Lee, Nicholas M Boffi, Seunghoon Hong, and Jinwoo Kim. Self-conditioned flow map language models via fixed-point flows. *arXiv preprint arXiv:2607.00714*, 2026.

[78] Zeyu Michael Li, William Xingxu Chen, Bingshuo Qian, Jiayin Liu, and Xiang Cheng. Elf-reg: Scaling continuous diffusion language models to reasoning tasks. *arXiv preprint arXiv:2609.29102*, 2026.

[79] Hongyuan Tao, Xinggang Wang, Lianghui Zhu, Yongkang Li, Yunchao Wei, Bin Feng, Shaoyu Chen, Qian Zhang, Chang Huang, and Kai Yu. Multimodal flow: Unified flow modeling of language and vision in embedding spaces. *arXiv preprint arXiv:2609.40362*, 2026.

[80] Jiaxiu Jiang, Jingjing Ren, Wenbo Li, Bo Wang, Haoze Sun, Yijun Yang, Jianhui Liu, Yanbing Zhang, Shenghe Zheng, Yuan Zhang, et al. Textldm: Language modeling with continuous latent diffusion. *arXiv preprint arXiv:2605.07748*, 2026.

[81] Jiajun Liang, Yucheng Liao, Yukang Cao, Jiazhe Wei, Ken Li, Wende Tan, Jiankun Zhang, ZY Cui, Jingkang Yang, Liucheng Guo, et al. Aurora-lm: Autoencoding unified representation for continuous-latent diffusion language modeling. *arXiv preprint arXiv:2608.02602*, 2026.

[82] Junzhe Shen, Jieru Zhao, Ziwei He, and Zhouhan Lin. Codar: Continuous diffusion language models are more powerful than you think. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 23172–23187, 2026.

[83] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.

[84] Zekai Zhang, Yunjie Tian, Yanjin He, Xiaoyan Zhang, Dongdi Zhao, Qing Qu, and Di Fu. Scaling and distilling text embeddings for better diffusibility. *arXiv preprint arXiv:2610.01016*, 2026.

[85] Biao Zhang, Paul Suganthan, Gaël Liu, Ilya Philippov, Sahil Dua, Ben Hora, Kat Black, Gus Martins, Omar

Sanseviero, Shreya Pathak, et al. T5gemma 2: Seeing, reading, and understanding longer. *arXiv preprint arXiv:2512.14856*, 2025.

[86] Hongcan Guo, Qinyu Zhao, Yian Zhao, Shen Nie, Rui Zhu, Qiushan Guo, Feng Wang, Tao Yang, Hengshuang Zhao, Guoqiang Wei, et al. Continuous latent diffusion language model. *arXiv preprint arXiv:2605.06548*, 2026.

[87] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.

[88] Yizhe Zhang, Jiatao Gu, Zhuofeng Wu, Shuangfei Zhai, Joshua Susskind, and Navdeep Jaitly. Planner: Generating diversified paragraph via latent language diffusion model. *Advances in Neural Information Processing Systems*, 36:80178–80190, 2023.

[89] Viacheslav Meshchaninov, Egor Chimbulatov, Alexander Shabalin, Aleksandr Abramov, and Dmitry Vetrov. Cosmos: Compressed and smooth latent space for text diffusion modeling. *Advances in Neural Information Processing Systems*, 38:14271–14299, 2025.

[90] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, pages 23716–23736, 2022.

[91] Yulin Yuan, Ying Zhang, and Xiangming Meng. One latent, many tokens: Jointly learning compressed embeddings for efficient language diffusion. *arXiv preprint arXiv:2609.33698*, 2026.

[92] ZiYi Dong, Yuliang Huang, Weijian Deng, Xiangyang Ji, Liang Lin, and Pengxu Wei. Language generation as optimal control: Closed-loop diffusion in latent control space. *arXiv preprint arXiv:2605.14531*, 2026.

[93] Chenze Shao, Darren Li, Fandong Meng, and Jie Zhou. Continuous autoregressive language models. *arXiv preprint arXiv:2510.27688*, 2025.

[94] Hongcan Guo, Yian Zhao, Feng Wang, Shen Nie, Guoqiang Wei, Yan Zeng, and Hengshuang Zhao. Cola2: Co-adaptive hierarchical language modeling with continuous semantic compression and balanced discrete generation. *arXiv preprint arXiv:2610.31305*, 2026.

[95] Xinsong Feng, Peng Du, Zhizhuo Yang, Daniel M Bikel, Jiayun Wang, and Haipeng Chen. Denoising blocks, not tokens: Efficient compressed continuous diffusion with branching token realization. *arXiv preprint arXiv:2610.09311*, 2026.

[96] Xiaochen Zhu, Georgi Karadzhov, Chenxi Whitehouse, and Andreas Vlachos. Segment-level diffusion: A framework for controllable long-form generation with diffusion language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4163–4183, 2025.

[97] Justin Lovelace, Varsha Kishore, Yiwei Chen, and Kilian Weinberger. Diffusion guided language modeling. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14936–14952, 2024.

[98] Loïc Barrault, Paul-Ambroise Duquenne, Maha Elbayad, Artyom Kozhevnikov, Belen Alastruey, Pierre Andrews, Mariano Coria, Guillaume Couairon, Marta R. Costa-jussà, David Dale, et al. Large concept models: Language modeling in a sentence representation space. *arXiv preprint arXiv:2412.08821*, 2024.

[99] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. Sonar: sentence-level multimodal and language-agnostic representations. *arXiv preprint arXiv:2308.11466*, 2023.

[100] Jinjin He, Xinsong Feng, Lucheng Fu, Sinan Wang, Bo Zhu, and Jiayun Wang. Rethinking the roles of diffusion and decoding: Predictive-state flows for parallel language generation. *arXiv preprint arXiv:2610.12898*, 2026.

[101] Roy Uziel, Omer Belhasin, Itay Levy, Akhiad Bercovich, Ran El-Yaniv, Ran Zilberstein, and Michael Elad. Crocodil: Continuous and robust conditioned diffusion for language. *arXiv preprint arXiv:2603.20210*, 2026.

[102] Alex Graves, Rupesh Kumar Srivastava, Timothy Atkinson, and Faustino Gomez. Bayesian flow networks. *arXiv preprint arXiv:2308.07037*, 2023.

[103] Justin Deschenaux, Alexandre Galashov, Andrew Campbell, Li Kevin Wenliang, James Thornton, Arnaud Doucet, and Valentin De Bortoli. Simplex diffusion models. *arXiv preprint arXiv:2609.35553*, 2026.

[104] Jaehyeong Jo and Sung Ju Hwang. Continuous diffusion model for language modeling. In *Advances in Neural Information Processing Systems*, pages 97394–97430, 2025.

[105] Jannis Chemseddine, Gregor Kornhardt, and Gabriele Steidl. Spherical flows for sampling categorical data. *arXiv preprint arXiv:2605.05629*, 2026.

[106] Andrea Miele and Ilija Bogunovic. Spherical brownian diffusion for language modeling. *arXiv preprint arXiv:2610.35101*, 2026.

[107] Zhicheng Du and Lan Ma. Continuous language diffusion as a decoder-interface problem. *arXiv preprint arXiv:2606.08810*, 2026.

[108] Oscar Davis, Daan Roos, Alice Bizeul, Amitis Shidani, Victor Turrisi, Louis Béthune, Pierre Ablin, and Marco Cuturi. On the design space of continuous diffusion language models. *arXiv preprint arXiv:2610.13834*, 2026.

[109] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[110] Arindam Banerjee, Xin Guo, and Hui Wang. On the optimality of conditional expectation as a bregman predictor. *IEEE Transactions on Information Theory*, 51 (7):2664–2669, 2005.

[111] Pavel Golik, Patrick Doetsch, and Hermann Ney. Cross-entropy vs. squared error training: a theoretical and experimental comparison. In *Proc. Interspeech 2013*, pages 1756–1760, 2013.

[112] Theodoros Kouzelis, Spyros Gidaris, and Nikos Komodakis. Self-regularized flow language models. *arXiv preprint arXiv:2610.37115*, 2026.

[113] Na Li, Yuchen Jiao, Changxiao Cai, and Gen Li. Convergeflow: Language flow with provable convergence

to token embeddings. *arXiv preprint arXiv:2608.23551*, 2026.

[114] Diederik P. Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. In *Advances in Neural Information Processing Systems*, pages 21696–21707, 2021.

[115] Arash Vahdat, Karsten Kreis, and Jan Kautz. Score-based generative modeling in latent space. *Advances in neural information processing systems*, 34:11287–11302, 2021.

[116] Gabriel Raya, Bac Nguyen, Georgios Batzolis, Yuhta Takida, Dejan Stancevic, Naoki Murata, Chieh-Hsin Lai, Yuki Mitsufuji, and Luca Ambrogioni. Noise scheduling as information-guided allocation in diffusion training. *arXiv preprint arXiv:2602.18647*, 2026.

[117] Mingyu Jo, Jaesik Yoon, Justin Deschenaux, Caglar Gulcehre, and Sungjin Ahn. Loopholing discrete diffusion: Deterministic bypass of the sampling wall. In *International Conference on Learning Representations*, volume 2026, pages 93890–93918, 2026.

[118] Yuxuan Liu, Tianchi Yang, Shaohan Huang, Zihan Zhang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, and Qi Zhang. Text diffusion with reinforced conditioning. *arXiv preprint arXiv:2402.14843*, 2024.

[119] Dat Nguyen-Cong, Tung Kieu, and Hoang Thanh-Tung. Fastdiss: Few-step match many-step diffusion language model on sequence-to-sequence generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 17572–17592, 2026.

[120] Alexander Shabalin, Simon Elistratov, Viacheslav Meshchaninov, Ildus Sadrtdinov, and Dmitry Vetrov. Why gaussian diffusion models fail on discrete data and how to prevent it? *arXiv preprint arXiv:2604.02028*, 2026.

[121] Shuai Zhang, Zijie Chen, Huachuan Qiu, Hongliang He, Lun Du, and Zhenzhong Lan. A dominant self-conditioning direction drives repetition in unconditional continuous diffusion language models. *arXiv preprint arXiv:2607.00588*, 2026.

[122] Alec Helbling, Andrey Bryutkin, Mauro Martino, Duen Horng Chau, Nima Dehmamy, and Hendrik Strobelt. Flow reasoning models: Turning flows into efficient recurrent reasoners. *arXiv preprint arXiv:2606.29150*, 2026.

[123] Ayhan Suleymanzade, Chanhyuk Lee, Floor Eijkelboom, Nicholas M Boffi, İsmail İlkan Ceylan, and Jinwoo Kim. Thinking with looped flows. *arXiv preprint arXiv:2609.11801*, 2026.

[124] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4172–4182. IEEE, 2023.

[125] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[126] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, pages 3942–3951, 2018.

[127] Weijian Luo, Yujian Chen, Hao Jiang, Guang Cheng, and Tianyang Hu. Designing transformers for continuous diffusion language models. *arXiv preprint arXiv:2610.22330*, 2026.

[128] Xiaochuang Han, Sachin Kumar, Yulia Tsvetkov, and Marjan Ghazvininejad. David helps goliath: Inference-time collaboration between small specialized and large general diffusion lms. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8385–8400, 2024.

[129] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *Advances in neural information processing systems*, 37:115491–115522, 2024.

[130] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, volume 2025, pages 29687–29707, 2025.

[131] Vassilis Papadopoulos, Jérémie Wenger, and Clément Hongler. Arrows of time for large language models. In *International Conference on Machine Learning*, pages 39509–39528, 2024.

[132] Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *International Conference on Machine Learning*, pages 30749–30768, 2025.

[133] Tong Wu, Zhihao Fan, Xiao Liu, Hai-Tao Zheng, Yeyun Gong, Yelong Shen, Jian Jiao, Juntao Li, Zhongyu Wei, Jian Guo, Nan Duan, and Weizhu Chen. AR-diffusion: Auto-regressive diffusion model for text generation. *Advances in Neural Information Processing Systems*, 36:39957–39974, 2023.

[134] Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *Advances in Neural Information Processing Systems*, 37:24081–24125, 2024.

[135] Yair Schiff, Omer Belhasin, Roy Uziel, Matan Rusanovsky, Ran Zilberstein, Marianne Arriola, Gilad Turok, Guanghan Wang, Volodymyr Kuleshov, and Michael Elad. Clock diffusion: Efficient semi-autoregressive continuous diffusion language models. *arXiv preprint arXiv:2610.00894*, 2026.

[136] Parikshit Bansal and Sujay Sanghavi. Token time continuous diffusion for language modeling. *arXiv preprint arXiv:2607.14106*, 2026.

[137] Chanhyuk Lee, Wonjung Kim, Patrick Pynadath, Jaehoon Yoo, Satoshi Hayakawa, Ruqi Zhang, Jinwoo Kim, and Seunghoon Hong. Asynchronous flow map language models. *arXiv preprint arXiv:2610.42018*, 2026.

[138] Bocheng Li, Zhujin Gao, and Linli Xu. Unifying continuous and discrete text diffusion with non-simultaneous diffusion processes. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11530–11551, 2025.

[139] Iskander Azangulov, Kianoosh Ashouritaklimi, Leo Zhang, Simon Vary, and Patrick Rebeschini. Masked language flow models. *arXiv preprint arXiv:2606.27617*, 2026.

[140] Pascal Jutras-Dubé, Patrick Pynadath, Jeremy Lu, Yuan Gao, and Ruqi Zhang. Sticky jump diffusions: A unifying framework for discrete, continuous, and hybrid diffusion. In *Advances in Neural Information Processing Systems*, 2026.

[141] Sophia Tang and Pranam Chatterjee. Expanding flow maps. *arXiv preprint arXiv:2607.21585*, 2026.

[142] Hui Ren, Zihan Li, Chang Liu, Huidong Liu, and Alexander Schwing. Hierarchical continuous diffusion language models. *arXiv preprint arXiv:2610.02193*, 2026.

[143] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models. In *International Conference on Learning Representations*, 2021.

[144] Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. In *European Conference on Computer Vision*, pages 23–40. Springer, 2024.

[145] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models. *Machine Intelligence Research*, 22(4):730–751, 2025. ISSN 2731-5398. doi: 10.1007/s11633-025-1562-4.

[146] Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. Diffuseq-v2: Bridging discrete and continuous text spaces for accelerated seq2seq diffusion models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9868–9875, 2023.

[147] Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. Guiding a diffusion model with a bad version of itself. *Advances in Neural Information Processing Systems*, 37: 52996–53021, 2024.

[148] Rohit Dilip, Tianrong Chen, Yuyang Wang, David Van Valen, Josh Susskind, and Miguel Angel Bautista. How to guide your language flow. *arXiv preprint arXiv:2609.19356*, 2026.

[149] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. In *Advances in Neural Information Processing Systems*, pages 75460–75482, 2025.

[150] Zhengyang Geng, Yiyang Lu, Zongze Wu, Eli Shechtman, J Zico Kolter, and Kaiming He. Improved mean flows: On the challenges of fastforward generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 30467–30476, 2026.

[151] Jaesung Tae, Hamish Ivison, Sachin Kumar, and Arman Cohan. Tess 2: A large-scale generalist diffusion language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21171–21188, 2025.

[152] Jinwoo Kim, Taylor Berg-Kirkpatrick, and Loris D'Antoni. Continuous diffusion models can obey formal syntax. In *International Conference on Machine Learning*, pages 57136–57151, 2026.

[153] Daisuke Oba, Hiroki Furuta, and Naoaki Okazaki. Bct: Bayes-consistent tempering for flow language models. *arXiv preprint arXiv:2610.27641*, 2026.

[154] Bocheng Li and Linli Xu. Improving few-step language flows with untied self-conditioning. *arXiv preprint arXiv:2608.22244*, 2026.

[155] Andrei Rekesh, Jarrid Rector-Brooks, and Cheng-Hao Liu. Inverse-confidence sampling for continuous diffusion language models. In *ICML 2026 Workshop on Foundations of Deep Generative Models: Understanding Memorization, Generalization, and Reasoning*, 2026.

[156] Haoning Yang and Lili Pan. Graft: Restoring the missing sampling interface in continuous diffusion language models. *arXiv preprint arXiv:2610.29136*, 2026.

[157] Weitian Wang, Lianlei Shan, Shubham Rai, Cecilia De La Parra, and Akash Kumar. x-prediction flow: Efficient continuous decoding for masked diffusion language models. *arXiv preprint arXiv:2606.29066*, 2026.

[158] Sofia Maria Lo Cicero Vaina, Nikita Balagansky, and Daniil Gavrilov. Diffusion language models generation can be halted early. *arXiv preprint arXiv:2305.10818*, 2023.

[159] Jiasheng Ye, Zaixiang Zheng, Yu Bao, Lihua Qian, and Mingxuan Wang. DINOISER: Diffused Conditional Sequence Learning by Manipulating Noises. *arXiv preprint arXiv:2302.10025*, 2024.

[160] Shankar Kumar and Bill Byrne. Minimum bayes-risk decoding for statistical machine translation. In *Proceedings of the human language technology conference of the North American chapter of the association for computational linguistics: HLT-NAACL 2004*, pages 169–176, 2004.

[161] Bryan Eikema and Wilker Aziz. Is map decoding all you need? the inadequacy of the mode in neural machine translation. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 4506–4520, 2020.

[162] Guangfu Guo, Xiaoqian Lu, Linsey Pang, Weiran Yao, Haolin Chen, Long Cheng, and Kunpeng Liu. Rangeswa: Training-free length extrapolation for continuous diffusion language models. *arXiv preprint arXiv:2610.42084*, 2026.

[163] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International conference on machine learning*, pages 19274–19286. PMLR, 2023.

[164] Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning*, pages 15343–15355, 2026.

[165] Jeff Shen, Zhejian Zhou, Ryan Lee, Jonathan May, and Shirley Ho. Flashmap: Flow map speculative decoding. *arXiv preprint arXiv:2610.24365*, 2026.

[166] Vincent Tao Hu, Di Wu, Yuki M. Asano, Pascal Mettes, Basura Fernando, Björn Ommer, and Cees G. M. Snoek. Flow matching for conditional text generation in a few sampling steps. In *Proceedings of the 18th*

Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers), pages 380–392, 2024.

[167] Pan Liu, Xiaohua Tian, and Zhouhan Lin. Enable fast sampling for seq2seq text diffusion. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8495–8505, 2024.

[168] Shujian Zhang, Lemeng Wu, Chengyue Gong, and Xingchao Liu. LanguageFlow: Advancing diffusion language generation with probabilistic flows. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3893–3905, 2024.

[169] Runzhe Zhang, Letian Chen, Wenpeng Zhang, Zhouhan Lin, and Peilin Zhao. FlowLM: Few-step language modeling via diffusion-to-flow adaptation. *arXiv preprint arXiv:2605.20199*, 2026.

[170] Nicholas M Boffi, Michael S Albergo, and Eric Vanden-Eijnden. Flow map matching with stochastic interpolants: A mathematical framework for consistency models. *Transactions on Machine Learning Research*, 2025.

[171] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning*, pages 32211–32252, 2023.

[172] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations*, 2022.

[173] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ODE trajectory of diffusion. In *International Conference on Learning Representations*, pages 44493–44525, 2024.

[174] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. In *International Conference on Learning Representations*, pages 34668–34684, 2025.

[175] Linqi Zhou, Mathias Parger, Ayaan Haque, and Jiaming Song. Terminal velocity matching. In *International Conference on Learning Representations*, volume 2026, pages 87506–87530, 2026.

[176] Maksim Semenov, Nikita Balagansky, Ilya Drobyshevskiy, Dmitry Baranchuk, Nikita Gushchin, Daniil Gavrilov, Aibek Alanov, and Viacheslav Meshchaninov. Latent space co-training for few-step text generation. In *NeurIPS 2026 Workshop on Beyond Next-Token Prediction: Diffusion and Flow Models for Next-Generation Decoding*, 2026.

[177] Tianqi Chen, Shujian Zhang, and Mingyuan Zhou. DLM-One: Diffusion language models for one-step sequence generation. *arXiv preprint arXiv:2506.00290*, 2025.

[178] Fred Zhangzhi Peng, Avishek Joey Bose, Anru R. Zhang, and Alexander Tong. Coupling models for one-step discrete generation. *arXiv preprint arXiv:2605.07193*, 2026.

[179] David Li, Chanhyuk Lee, Jaehoon Yoo, Nikita Gushchin, Floor Eijkelboom, Eric Moulines, Maxim Panov, Alexander Korotin, Seunghoon Hong, and Jinwoo Kim. One-step generation via Riemannian Wasserstein gradient flows. *arXiv preprint arXiv:2610.04454*, 2026.

[180] Jiayu Liu, Ruizhi Zhao, Tianyun Ji, Yee Whye Teh, Zhenya Huang, and Ning Miao. One-step text generation by seq-drifting. *arXiv preprint arXiv:2610.12879*, 2026.

[181] Felix Draxler, Justus Will, Farrin Marouf Sofian, Theofanis Karaletsos, Sameer Singh, and Stephan Mandt. Parallel token prediction for language models. In *International Conference on Learning Representations*, pages 5624–5652, 2026.

[182] Chi Zhang, Xixi Hu, Bo Liu, and Qiang Liu. Gumbel distillation for parallel text generation. In *International Conference on Learning Representations*, pages 153642–153671, 2026.

[183] Cheuk-Kit Lee, Florentin Coeurdoux, Yuyuan Chen, Sophia Tang, Peter Potaptchik, Yilun Du, Michael S Albergo, and Eric Vanden-Eijnden. Beckmann transport models: From autonomous flows to one-step maps. *arXiv preprint arXiv:2608.01692*, 2026.

[184] Zhengyang Geng, Ashwini Pokle, Weijian Luo, Justin Lin, and Zico Kolter. Consistency models made easy. In *International Conference on Learning Representations*, volume 2025, pages 75158–75185, 2025.

[185] Yeongmin Kim, Arnaud Doucet, Andrew Campbell, Valentin De Bortoli, Thomas Mensink, and David Ruhe. Gumbel straight flow: Distilling autoregressive models into one-step flow maps. *arXiv preprint arXiv:2610.00497*, 2026.

[186] Manan Agarwal, Sheel Shah, Chanhyuk Lee, Jaehoon Yoo, Jerry Huang, Seunghoon Hong, Aditi Raghunathan, Jinwoo Kim, and Nicholas M Boffi. Posterior refinement: Fast language generation via any-order flow maps. *arXiv preprint arXiv:2606.24773*, 2026.

[187] Neta Shaul, Chao Liu, Arash Vahdat, and Julius Berner. Parallel decoding distillation for fast image and video generation. *arXiv preprint arXiv:2607.26004*, 2026.

[188] Jedrzej Warczyński, Ondřej Dušek, and Mateusz Lango. One-step nonautoregressive natural language generation with shortcut flow matching models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 646–655, 2026.

[189] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T. Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6613–6623, 2024.

[190] Mingyuan Zhou, Huangjie Zheng, Zhendong Wang, Mingzhang Yin, and Hai Huang. Score identity distillation: Exponentially fast distillation of pretrained diffusion models for one-step generation. In *International Conference on Machine Learning*, pages 62307–62331, 2024.

[191] Paul Le Van Kiem, Dario Shariatian, Umut Simsekli, and Alain Durmus. Distribution matching distillation for continuous diffusion language models. *arXiv preprint arXiv:2609.40235*, 2026.

[192] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3–4):229–256, 1992.

[193] Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting. *arXiv preprint arXiv:2602.04770*, 2026.

[194] Floor Eijkelboom, Nabil Iqbal, Max Welling, and Jan-Willem van de Meent. Categorical drifting models. In *ICML 2026 Workshop on Structured Probabilistic Inference & Generative Modeling*, 2026.

[195] Ilya Drobyshevskiy, Ilia Sudakov, Maksim Semenov, Denis Kuznedelev, Maksim Ignatov, Pavel Temirchev, Nikita Balagansky, Viacheslav Meshchaninov, Nikita Gushchin, and Dmitry Baranchuk. Representation-space mmd for diffusion language models. *arXiv preprint arXiv:2610.06648*, 2026.

[196] Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024.

[197] Baiyu Su, Runlong Liao, Lizhang Chen, and Qiang Liu. Rectified language flow. *arXiv preprint arXiv:2610.33811*, 2026.

[198] Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14297–14306, 2023.

[199] Zhicong Tang, Jianmin Bao, Dong Chen, and Baining Guo. Diffusion models without classifier-free guidance. *arXiv preprint arXiv:2502.12154*, 2025.

[200] Jerry Y Huang, Justin Lin, Sheel Shah, Kartik Nair, and Nicholas M Boffi. How to guide your flow: Few-step alignment via flow map reward guidance. In *International Conference on Machine Learning*, pages 47101–47152, 2026.

[201] Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. Accelerated sampling from masked diffusion models via entropy bounded unmasking. In *Advances in Neural Information Processing Systems*, pages 55981–56007, 2025.

[202] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. Maskgit: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11315–11325, 2022.

[203] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

[204] DiffusionGemma Team, Adrien Ali Taïga, James Assiene, Daniele Calandriello, Rahma Chaabouni, João Gante, Tamara von Glehn, Nate Keating, Chris Knutsen, Martin Kukla, et al. Diffusiongemma technical report. *arXiv preprint arXiv:2608.00146*, 2026.

[205] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

[206] Felix Draxler, Robin C Geyer, Daniel Geyfman, Farrin Marouf Sofian, Justus Will, Philipp Hildebrandt, Giosue Migliorini, Prakhar Srivastava, Gagan Sai Trinadh Duggaraju Peddiraju, Kushagra Pandey, et al. Parallel language models via variational decoding. *arXiv preprint arXiv:2610.26693*, 2026.

[207] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

[208] Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, Maksim Khadkevich, Nicolai Oswald, Enze Xie, Daniel Egert, Sharath Turuvekere Sreenivas, Shizhe Diao, et al. Nemotron-labs-diffusion: A tri-mode language model unifying autoregressive, diffusion, and self-speculation decoding. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 54868–54887, 2026.

[209] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[210] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[211] Longxuan Yu, Yunshu Wu, Yu Fu, Siheng Xiong, Rob Brekelmans, Hui Liu, Yue Dong, and Greg Ver Steeg. Dsl-llada: Scaling continuous denoising to 8b masked diffusion lms. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, pages 44727–44752, 2026.

[212] Zijian Jia, Peixuan Zhang, Genying Li, Di Wu, Yujie Fu, Si Li, Boxin Shi, and Shuchen Weng. Llaco: Continuous diffusion for large language models with competition-aware semantic scheduling. *arXiv preprint arXiv:2610.32604*, 2026.

[213] Nikhil Sardana, Jacob Portes, Sasha Doubov, and Jonathan Frankle. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. In *International Conference on Machine Learning*, pages 43445–43460, 2024.

[214] Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*, 2013.

[215] Aaron Gokaslan and Vanya Cohen. OpenWebText corpus. http://Skylion007.github.io/OpenWebTextCorpus, 2019.

[216] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *International Conference on Machine Learning*, pages 32819–32848, 2024.

[217] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked

diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. In *International Conference on Learning Representations*, pages 63186–63227, 2025.

[218] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. Generative frontiers: Why evaluation matters for diffusion language models. *arXiv preprint arXiv:2604.02718*, 2026.

[219] Antonio Franca and Alexander Tong. Hacking generative perplexity: Why unconditional text evaluation needs new metrics. *arXiv preprint arXiv:2606.08417*, 2026.

[220] Krishna Pillutla, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, and Zaid Harchaoui. Mauve: Measuring the gap between neural text and human text using divergence frontiers. *Advances in Neural Information Processing Systems*, 34: 4816–4828, 2021.

[221] Emiel Hoogeboom, David Ruhe, Jonathan Heek, Thomas Mensink, and Tim Salimans. Beyond single tokens: Distilling discrete diffusion models via discrete mmd. *arXiv preprint arXiv:2603.20155*, 2026.

[222] Gábor J Székely and Maria L Rizzo. Energy statistics: A class of statistics based on distances. *Journal of statistical planning and inference*, 143(8):1249–1272, 2013.

[223] Gregor Kornhardt, Moritz Piening, Jannis Chemseddine, and Gabriele Steidl. Sol: Measuring gaps between text distributions by double sliced wasserstein metrics. *arXiv preprint arXiv:2610.06513*, 2026.

[224] Bingbin Liu, Sebastien Bubeck, Ronen Eldan, Janardhan Kulkarni, Yuanzhi Li, Anh Nguyen, Rachel Ward, and Yi Zhang. Tinygsm: achieving >80% on gsm8k with small language models. *arXiv preprint arXiv:2312.09241*, 2023.

[225] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[226] Jaeyeon Kim, Jonathan Geuter, David Alvarez-Melis, Sham Kakade, and Sitan Chen. Stop training for the worst: Progressive unmasking accelerates masked diffusion training. In *International Conference on Machine Learning*, pages 57515–57538, 2026.

[227] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[228] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in neural information processing systems*, 36:21558–21572, 2023.

[229] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[230] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

[231] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

[232] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[233] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion transformers is easier than you think. In *International Conference on Learning Representations*, pages 87400–87442, 2025.

[234] Fred Zhangzhi Peng, Alexis Fox, Anru R Zhang, and Alexander Tong. Don't retrain, align: Adapting autoregressive lms to diffusion lms via representation alignment. *arXiv preprint arXiv:2605.06885*, 2026.

[235] Ge Wu, Shen Zhang, Ruijing Shi, Shanghua Gao, Zhenyuan Chen, Lei Wang, Zhaowei Chen, Hongcheng Gao, Yao Tang, Jian Yang, Ming-Ming Cheng, and Xiang Li. Representation entanglement for generation: Training diffusion transformers is much easier than you think. In *Advances in Neural Information Processing Systems*, pages 7714–7743, 2025.

[236] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.

[237] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[238] Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. Arc-agi-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

[239] Hanru Bai, Faissal Izermine, Oscar Davis, and T. Konstantin Rusch. Efficient reasoning with flow language models. *arXiv preprint arXiv:2610.09416*, 2026.

[240] Alankrita Bhatt, Mukur Gupta, Germain Kolossov, and Andrea Montanari. Generating from discrete distributions using diffusions: Insights from random constraint satisfaction problems. *arXiv preprint arXiv:2603.20589*, 2026.

[241] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

[242] Junyeob Baek, Mingyu Jo, Minsu Kim, Mengye Ren, Yoshua Bengio, and Sungjin Ahn. Generative recursive reasoning. *arXiv preprint arXiv:2605.19376*, 2026.

[243] Yueshan Huang, Gufeng Yu, Letian Chen, Haoyang Luan, and Yang Yang. Epic: multi-objective guided diffusion for epitope design in tcr-pmhc complexes. *Bioinformatics*, 42(6):btag358, 2026.

[244] Xinyou Wang, Zaixiang Zheng, Fei Ye, Dongyu Xue, Shujian Huang, and Quanquan Gu. Diffusion lan-

guage models are versatile protein learners. In *International Conference on Machine Learning*, pages 52309–52333, 2024.

[245] Sophia Tang, Yinuo Zhang, and Pranam Chatterjee. Peptune: De novo generation of therapeutic peptides with multi-objective-guided discrete diffusion. *International Conference on Machine Learning (ICML 2025)*, pages 59017–59065, 2025.

[246] Tong Chen, Yinuo Zhang, Sophia Tang, and Pranam Chatterjee. Multi-objective-guided discrete flow matching for controllable biological sequence design. *arXiv preprint arXiv:2505.07086*, 2025.

[247] Viacheslav Meshchaninov, Pavel Strashnov, Andrey Shevtsov, Fedor Nikolaev, Nikita Ivanisenko, Olga Kardymon, and Dmitry Vetrov. Diffusion on language model encodings for protein sequence generation. In *International Conference on Machine Learning*, pages 43837–43878, 2025.

[248] Zitai Kong, Yiheng Zhu, Yinlong Xu, Hanjing Zhou, Mingzhe Yin, Jialu Wu, Hongxia Xu, Chang-Yu Hsieh, Tingjun Hou, and Jian Wu. Protflow: Fast protein sequence design via flow matching on compressed protein language model embeddings. *arXiv preprint arXiv:2504.10983*, 2025.

[249] Chenyu Wang, Masatoshi Uehara, Yichun He, Amy Wang, Avantika Lal, Tommi Jaakkola, Sergey Levine, Aviv Regev, Hanchen Wang, and Tommaso Biancalani. Fine-tuning discrete diffusion models via reward optimization with applications to dna and protein design. In *International Conference on Learning Representations*, volume 2025, pages 47871–47899, 2025.

[250] Sophia Tang, Yuchen Zhu, Molei Tao, and Pranam Chatterjee. Tr2-d2: Tree search guided trajectory-aware fine-tuning for discrete diffusion. *arXiv preprint arXiv:2509.25171*, 2025.

[251] Kathleen M Chen, Aaron K Wong, Olga G Troyanskaya, and Jian Zhou. A sequence-based global map of regulatory activity for deciphering human genetics. *Nature genetics*, 54(7):940–949, 2022.

[252] Amaru Caceres Arroyo, Lea Bogensperger, Ahmed Allam, Michael Krauthammer, Konrad Schindler, and Dominik Narnhofer. Repurposing protein language models for latent flow-based fitness optimization. *arXiv preprint arXiv:2602.02425*, 2026.

[253] Seul Lee, Karsten Kreis, Srimukh Prasad Veccham, Meng Liu, Danny Reidenbach, Yuxing Peng, Saee Paliwal, Weili Nie, and Arash Vahdat. Genmol: A drug discovery generalist with discrete diffusion. In *International Conference on Machine Learning*, pages 33205–33226, 2025.

[254] Martin Vogt. Exploring chemical space—generative models and their evaluation. *Artificial Intelligence in the Life Sciences*, 3:100064, 2023.

[255] Xinyou Wang, Zaixiang Zheng, Fei Ye, Dongyu Xue, Shujian Huang, and Quanquan Gu. Dplm-2: A multimodal diffusion protein language model. In *International Conference on Learning Representations*, pages 35463–35490, 2025.

[256] Cheng-Yen Hsieh, Xinyou Wang, Daiheng Zhang, Dongyu Xue, Fei Ye, Shujian Huang, Zaixiang Zheng, and Quanquan Gu. Elucidating the design space of multimodal protein language models. In *International Conference on Machine Learning*, pages 24156–24175, 2025.

[257] Yinuo Zhang, Sophia Tang, Tong Chen, Elizabeth Mahood, Sophia Vincoff, and Pranam Chatterjee. Peptiverse: A unified platform for therapeutic peptide property prediction. *Nature Communications*, 17, 2026.

[258] Kristina Preuer, Philipp Renz, Thomas Unterthiner, Sepp Hochreiter, and Gunter Klambauer. Fréchet chemnet distance: a metric for generative models for molecules in drug discovery. *Journal of chemical information and modeling*, 58(9):1736–1741, 2018.

[259] Pavel Avdeyev, Chenlai Shi, Yuhao Tan, Kseniia Dudnyk, and Jian Zhou. Dirichlet diffusion score model for biological sequence generation. In *International Conference on Machine Learning*, pages 1276–1301. PMLR, 2023.

[260] Zhengqiang Zhang, Lingchen Sun, Rongyuan Wu, Qiaosi Yi, Xiangtao Kong, Chaodong Xiao, and Lei Zhang. Ldm-is-ae: Latent diffusion is an intrinsic auto-encoder for end-to-end image generation. In *Advances in Neural Information Processing Systems*, 2026.

[261] Bingliang Zhang, Wenda Chu, Yizhuo Li, Linjie Yang, Yisong Yue, Katherine Bouman, Yang Song, and Qiushan Guo. Speediff: Scalable pixel-anchored end-to-end latent diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 35893–35903, 2026.

[262] Xingjian Leng, Jaskirat Singh, Yunzhong Hou, Zhenchang Xing, Saining Xie, and Liang Zheng. Repae: Unlocking vae for end-to-end tuning with latent diffusion transformers. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 18262–18272. IEEE, 2025.

[263] Jonathan Heek, Emiel Hoogeboom, Thomas Mensink, and Tim Salimans. Unified latents (ul): How to train your latents. *arXiv preprint arXiv:2602.17270*, 2026.

[264] Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. Diffusion beats autoregressive in data-constrained settings. In *Advances in Neural Information Processing Systems*, pages 10581–10606, 2025.

[265] Jinjie Ni, Qian Liu, Longxu Dou, Chao Du, Zili Wang, Hang Yan, Tianyu Pang, and Michael Qizhe Shieh. Diffusion language models are super data learners. *arXiv preprint arXiv:2511.03276*, 2025.

[266] Sam McCallum, Zander W Blasingame, Timothy Herschell, Niklas Rindtorff, Alexander Tong, and James Foster. Strong stochastic flow maps. *arXiv preprint arXiv:2606.01086*, 2026.

[267] Zhengkai Pan, Peter Potaptchik, Wenxi Yao, Michael S Albergo, and Jakiw Pidstrigach. Itô maps for any-step sdes. *arXiv preprint arXiv:2606.11156*, 2026.

[268] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[269] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu,

Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[270] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[271] Xinyue Ai, Yutong Kelly He, Albert Gu, Russ Salakhutdinov, Zico Kolter, Nicholas Boffi, and Max Simchowitz. Joint distillation for fast likelihood evaluation and sampling in flow-based models. In *International Conference on Learning Representations*, volume 2026, pages 44940–44977, 2026.

[272] RuiKang OuYang, Hanlin Yu, Xinyue Ai, Yutong He, Nicholas M Boffi, Pradeep Ravikumar, Jose Miguel Hernandez-Lobato, Max Simchowitz, Benjamin Kurt Miller, and Omar Chehab. Few-step boltzmann generators via scalable likelihood flow maps. *arXiv preprint arXiv:2606.29110*, 2026.

[273] Francesco M. Ruscio, Timothy James Hitge, Fred Zhangzhi Peng, Danyal Rehman, Zander W. Blasingame, Nicholas M. Boffi, Michael M. Bronstein, Alexander Tong, and Avishek Joey Bose. Normalizing flow maps. *arXiv preprint arXiv:2610.22711*, 2026.

[274] Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-grpo: Training flow matching models via online rl. In *Advances in Neural Information Processing Systems*, pages 40783–40818, 2025.

[275] Zhiqi Li, Wen Zhang, and Bo Zhu. Flow-map grpo: Reinforcement learning for few-step flow-map generators via anchored stochastic composition. *arXiv preprint arXiv:2607.00535*, 2026.

[276] Abbas Mammadov, Jerry Y Huang, Justin Lin, Partha Kaushik, Sheel Shah, Kartik Nair, Yee Whye Teh, and Nicholas M Boffi. Wtf?! simulation-free reinforcement learning with wasserstein-tilted flow maps. In *Advances in Neural Information Processing Systems*, 2026.

[277] Xu Han, Jiajing Hu, and Li-Ping Liu. Backpropagating verifier rewards through continuous flow language models. *arXiv preprint arXiv:2610.19080*, 2026.

[278] Georgios Batzolis, Georgios Kontos, Bruno Trentini, Pranam Chatterjee, Micha Livne, Mark Girolami, and Luca Ambrogioni. Multimodal co-generation with continuous bitstream diffusion. *arXiv preprint arXiv:2610.14349*, 2026.

[279] Jiahan Li, Huangyuan Su, Shengbang Tong, Shusheng Yang, Ruihan Guo, Suozhi Huang, Hexu Zhao, Pratyusha Sharma, Sherry Yang, Jiatao Gu, Rob Fergus, and Kyunghyun Cho. Juice: Unified multimodal modeling in pretrained representation spaces. *arXiv preprint arXiv:2610.28801*, 2026.

[280] Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. In *International Conference on Learning Representations*, volume 2025, pages 6446–6469, 2025.

[281] Yoshua Bengio, Geoffrey Hinton, Andrew Yao, Dawn Song, Pieter Abbeel, Trevor Darrell, Yuval Noah Harari, Ya-Qin Zhang, Lan Xue, Shai Shalev-Shwartz, et al. Managing extreme ai risks amid rapid progress. *Science*, 384(6698):842–845, 2024.

[282] Tomek Korbak, Mikita Balesni, Elizabeth Barnes, Yoshua Bengio, Joe Benton, Joseph Bloom, Mark Chen, Alan Cooney, Allan Dafoe, Anca Dragan, et al. Chain of thought monitorability: A new and fragile opportunity for ai safety. *arXiv preprint arXiv:2507.11473*, 2025.